

FIN BANK ATM SIMULATOR

CI453 Group Assignment 1 — Software Development Report

Group Fri-C107-19

Name	Student Number	Role & Contribution
Daniella — Lead Developer	25817373	Project Lead; PrimeAccount, SavingAccount, Transfer, Bug Fixes, View.java, Light/Dark Mode, Welcome/Goodbye Screens, code coordination
Reema — Project Manager, Core Developer	25838268	Project Manager; Create Account, Temp Lock, Audit Log, DataManager, Large Transaction Confirmation, Auto-Save, Last Login, Code coordination
Haaroun — Security & QA	25811005	UIModel State Machine, Login Security, Change Password, JUnit Testing
Amr — Research & UI	25800030	Business Research (ATM procurement & FinTech market); UI/UX Design, JUnit Testing

Lab Tutor: Shan Shine

Academic Year 2025/26 | Submission Date: 06/05/2026

Module: CI453 — Working In The Computing Industry

Table of Contents

Table of Contents.....	2
1. Understanding the FinTech Sector.....	4
1.1 Secure Transactions & User Authentication.....	4
1.2 Data Management & Cybersecurity.....	4
2. Introduction	5
3. Process	6
3.1 Development Methodology	6
3.2 Team Roles and Responsibilities	6
3.3 Tools and Environment.....	7
4. Application Description	8
4.1 Architecture — MVC Pattern.....	8
4.2 Class Overview	8
4.3 Key Features — Description and Screenshots	10
4.4 Automated Testing — JUnit 5.....	19
5. Legal and Ethical Issues.....	21
5.1 Data Protection and GDPR.....	21
5.2 Computer Misuse Act 1990.....	21
5.3 Ethical Considerations	21
5.4 Accessibility.....	21
5.5 Social Considerations	21
6. Research.....	23
6.1 What Banks Look For in ATM Software Providers	23
6.2 Computing Career Opportunities in FinTech	24
6.3 Future Trends in FinTech.....	24
7. Critical Review	26
7.1 Three Strengths	26
7.2 Four Things We Would Change	26
8. Statement of Individual Contribution - <i>Written individually</i>	28
8.1 Daniella — Lead Developer (Student No. 25817373).....	28
8.2 Reema — Project Manager, Core Developer (Student No. 25838268)	28
8.3 Haaroun — Security and QA (Student No. 25811005)	28
8.4 Amr — Research and UI (Student No. 25800030).....	29
9. References.....	30
10. Appendices	31
Appendix A — Test Account Credentials	31
Appendix B — GitHub Repository	31
Appendix C — Class Diagram	32
Appendix D — JUnit Test Results	33

Appendix E — Sample audit.log Output	33
Appendix F — Key Code Implementations.....	33

1. Understanding the FinTech Sector

1.1 Secure Transactions & User Authentication

Financial Technology (FinTech) sits at the intersection of financial services and computing, using software, algorithms, and digital infrastructure to deliver faster, more accessible alternatives to traditional banking (Arner, Barberis and Buckley, 2015). At its core, every FinTech system must process transactions with atomicity and integrity — operations either complete fully or not at all. Our FIN Bank simulator enforces this through method-level validation across all financial operations, returning boolean results so the system never enters an inconsistent state. Identity verification is equally critical: our three-attempt timed lock prevents brute-force attacks while aligning with PCI DSS Requirement 8 on access control, and a last-attempt warning mirrors real ATM behaviour (Nasir et al., 2023).

1.2 Data Management & Cybersecurity

Robust data management and cybersecurity are inseparable from any FinTech deployment. Our Transaction class captures a structured record of every financial event, powering the Mini Statement and feeding our AuditLogger. This writes tamper-evident entries to audit.log in line with FCA Transaction Reporting rules. DataManager auto-saves account state after every transaction, ensuring data survives a restart. Against cybersecurity threats — which disproportionately target financial platforms — our application applies PIN masking, MVC separation to prevent direct balance manipulation from the View layer, and persistent audit logging (Nasir et al., 2023).

2. Introduction

This report documents the design, development, and extension of the FIN Bank ATM Simulator, a JavaFX desktop application built upon a provided prototype codebase. Following the Model-View-Controller (MVC) architectural pattern, our group substantially extended the application to produce a professional-grade FinTech demonstration tool. The final application supports:

- Dynamic account creation — users can register a new account at the ATM without any hardcoded data
- Login with PIN authentication, last-attempt warning, and a 5-minute temporary lock after 3 failures
- Four account types: Standard, Prime, Student (daily limit), and Saving (interest)
- Balance enquiry with last login timestamp displayed
- Cash withdrawal with large transaction confirmation (over £500)
- Deposit two-step fund transfer between accounts
- Secure two-step password change flow
- Mini statement: last 5 transactions with masked account numbers
- Currency exchange: 5 currencies (USD, EUR, JPY, AED, CAD)
- Interest application for Saving accounts
- Auto-save: account data persisted after every transaction via DataManager
- Audit log: all actions written to audit.log via AuditLogger
- Liquid Glass UI: animated mesh gradient, holographic button hover, light/dark mode toggle
- Welcome and goodbye screens with animated transitions
- JUnit 5 automated test suite covering all core business logic

A short video demonstration is available at:

Demo Video: [Final finbank atm demo.mov](#)

3. Process

3.1 Development Methodology

The team adopted an approach structured around weekly goals. A shared brainstorming document coordinated feature ideas, task assignments, and progress tracking. Version control was managed via GitHub, with each member working on their own named branch before merging into main.

3.2 Team Roles and Responsibilities

Name	Student No.	Detailed Contribution
Daniella — Lead Developer	25817373	Overall code coordination and lead developer. Created PrimeAccount, SavingAccount (interest), Transfer feature. Authored View.java (mesh gradient, holographic buttons) and Controller. Managed merge conflicts and final code integration.
Reema — Project Manager	25838268	Project planning and developer. Implemented: Create Account feature, 5-minute Temporary Lock, Last Attempt Warning, AuditLogger.java, DataManager.java (auto-save), Large Transaction Confirmation, Mini Statement masked numbers, Last Login Time display on BAL, Managed merge conflicts.
Haaroun — Security & QA	25811005	UIModel finite-state machine design and implementation (9 states). Login attempt tracking, account locking, Change Password flow. Wrote JUnit tests with Amr.
Amr — Research & UI	25800030	Business research: ATM procurement market, what banks look for in ATM providers, FinTech trends. UI/UX: Light/Dark mode toggle, Welcome and Goodbye screen overlays. Wrote JUnit tests with Haaroun.

3.3 Tools and Environment

- IDE: IntelliJ IDEA Community Edition
- Language: Java 17 with JavaFX 17
- Build: Maven (pom.xml with JavaFX and JUnit 5 plugins)
- Testing: JUnit 5 (automated unit tests — see Section 4.4)
- Version Control: Git / GitHub
- Communication: Group WhatsApp, email and shared brainstorm document

4. Application Description

4.1 Architecture — MVC Pattern

The application strictly follows the Model-View-Controller (MVC) design pattern, separating concerns into three layers:

- Model (Bank, BankAccount, subclasses, Transaction, DataManager, AuditLogger): all business logic, financial calculations, data persistence, and audit logging.
- View (View.java): displays information and captures input. Never interacts with Bank directly — reads data via UIModel getters and forwards button presses to Controller.
- Controller (Controller.java): relay between View and UIModel. Receives a String action code per button press and calls the appropriate UIModel method.
- UIModel (UIModel.java): the ATM brain. Maintains a 9-state finite-state machine, coordinates Bank operations, and tells View to update after each action.

4.2 Class Overview

Class / File	Purpose	Author
Bank.java	Account storage (ArrayList), login/logout, transfer, currency exchange, login attempt tracking, timed lock	Daniella + Haaroun
BankAccount.java	Base class: withdraw, deposit, changePassword, transaction history, getMiniStatement(), last login time	Daniella + Reema
Transaction.java	Records one financial event (type, amount, balance, timestamp) for Mini Statement	Reema
StudentAccount.java	Extends BankAccount: £50 daily withdrawal limit via withdrawnToday counter	Reema
PrimeAccount.java	Extends BankAccount:	Daniella
SavingAccount.java	Extends BankAccount: 2% interest via addInterest(); interest preview before confirmation	Daniella

DataManager.java	Auto-saves account data to file after every transaction; loads data on startup	Reema
AuditLogger.java	Writes all actions (logins, transactions, errors) to audit.log with timestamps	Reema
UIModel.java	9-state FSM; all user action processing; coordinates Bank and View	Haaroun
Controller.java	Routes View button action codes to UIModel; handles Stmt, Cur, Int, CreateAcc, Confirm	Haaroun
View.java	Liquid Glass UI: animated mesh gradient, holographic buttons, light/dark mode, splash screens	Amr + Daniella
BankTest.java	JUnit 5 test suite: 20+ tests across all business logic classes	Amr

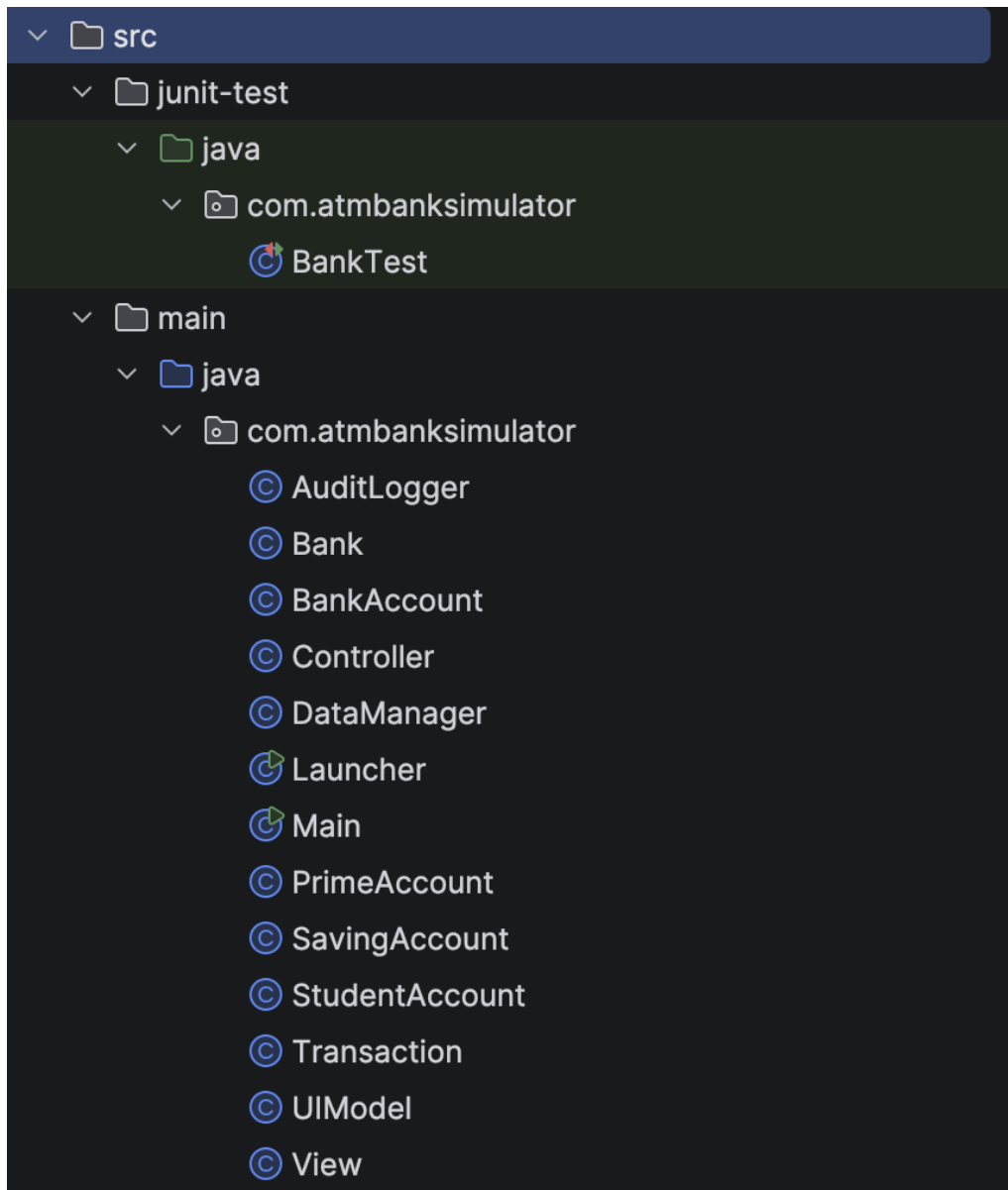


FIGURE: SCREENSHOT OF INTELLIJ PROJECT EXPLORER SHOWING THE FULL PACKAGE STRUCTURE

4.3 Key Features — Description and Screenshots

4.3.1 Welcome Screen and Animated Background

On launch, a glass card overlay fades in over an animated mesh gradient background driven by a JavaFX AnimationTimer. The user presses ENTER to transition to the main ATM.

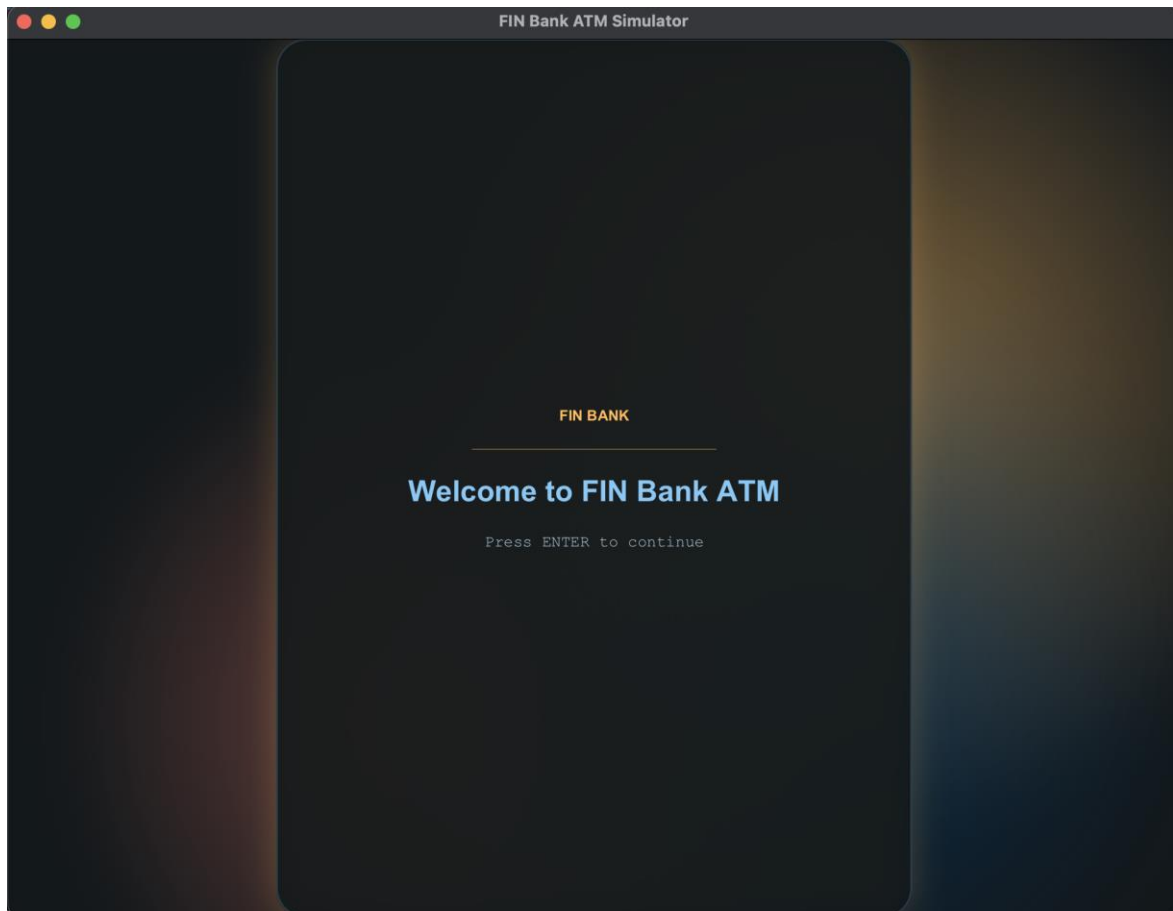


FIGURE: THE WELCOME SCREEN

4.3.2 Create Account

Users can register a new account directly at the ATM by selecting an account type, entering an account number, and setting a PIN — no hardcoded data required. The account is immediately added to Bank's ArrayList and saved via DataManager.

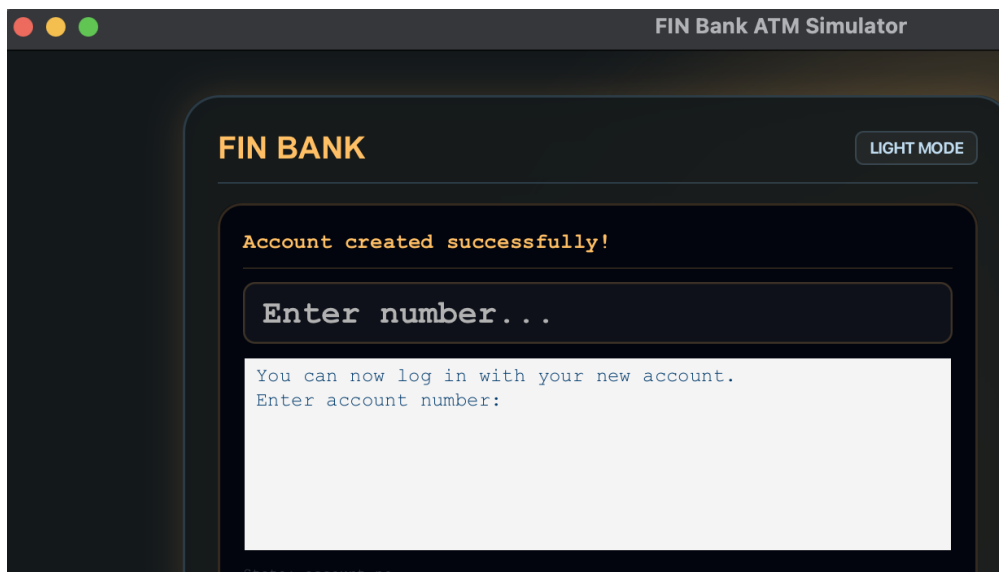
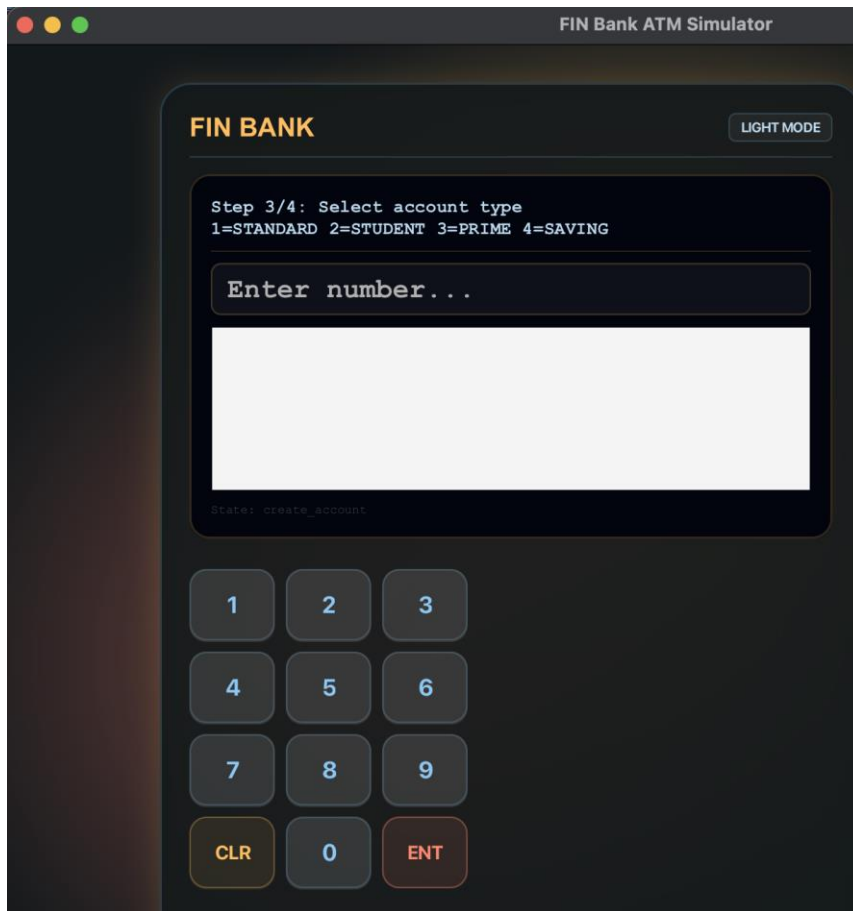


FIGURE: THE CREATE ACCOUNT SCREEN IN PROGRESS — SHOWS THE ACCOUNT TYPE SELECTION STEP AND THE SUCCESS MESSAGE.

4.3.3 Login Security — Last Attempt Warning and Timed Lock

After three consecutive failed PIN attempts the account locks for five minutes, with a warning displayed on the second failure. The account automatically unlocks after the timeout, balancing security against the ethical concern of permanently denying access to funds.

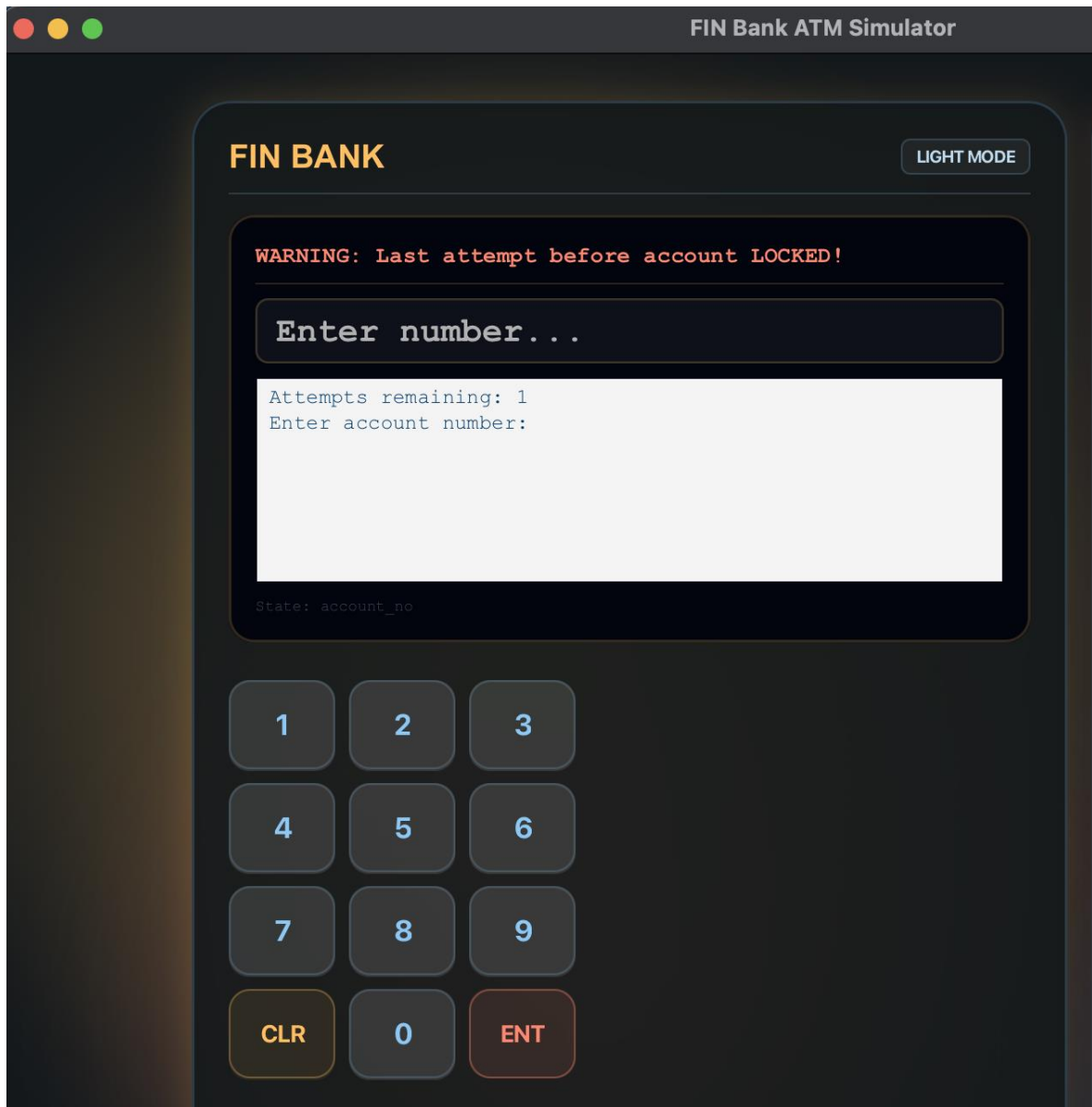


FIGURE: THE ERROR MESSAGE AFTER THE SECOND FAILED ATTEMPT, SHOWING THE LAST ATTEMPT WARNING

4.3.4 Large Transaction Confirmation

Any withdrawal over £500 requires a second ENT confirmation press before proceeding, reducing accidental or fraudulent high-value payment.

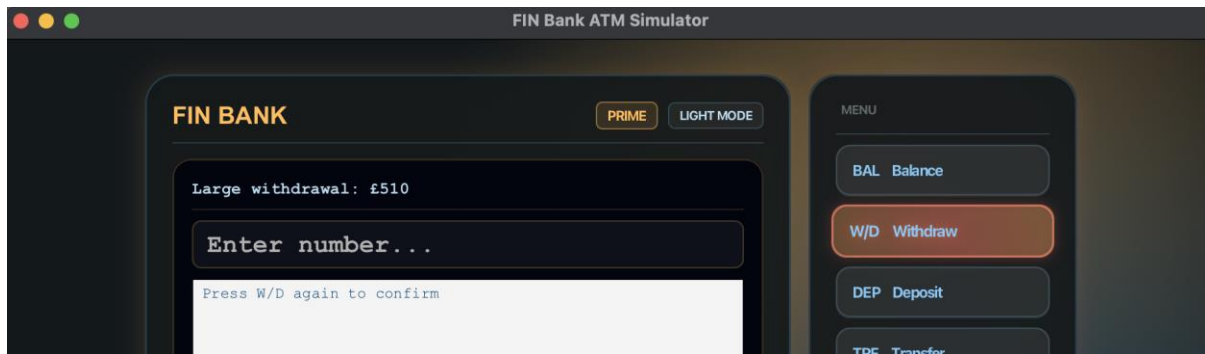


FIGURE: THE CONFIRMATION PROMPT SHOWING THE AMOUNT AND THE RE-CONFIRM INSTRUCTIONS

4.3.5 Mini Statement with Masked Account Number

The Mini Statement displays the last five transactions with the account number masked as **** to protect privacy under UK GDPR.

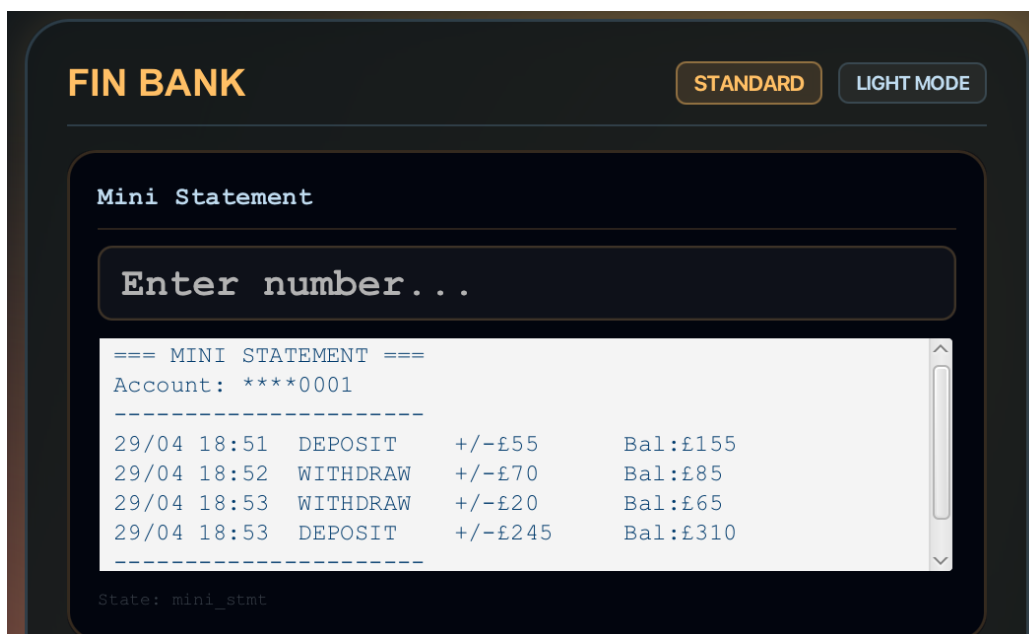


FIGURE: THE MINI STATEMENT SCREEN AFTER MAKING SEVERAL TRANSACTIONS — SHOWS THE MASKED ACCOUNT NUMBER AND LAST TRANSACTION ENTRIES.

4.3.6 Balance with Last Login Time

The balance screen shows the date and time of the user's previous login — a security feature that helps users detect unauthorised access.

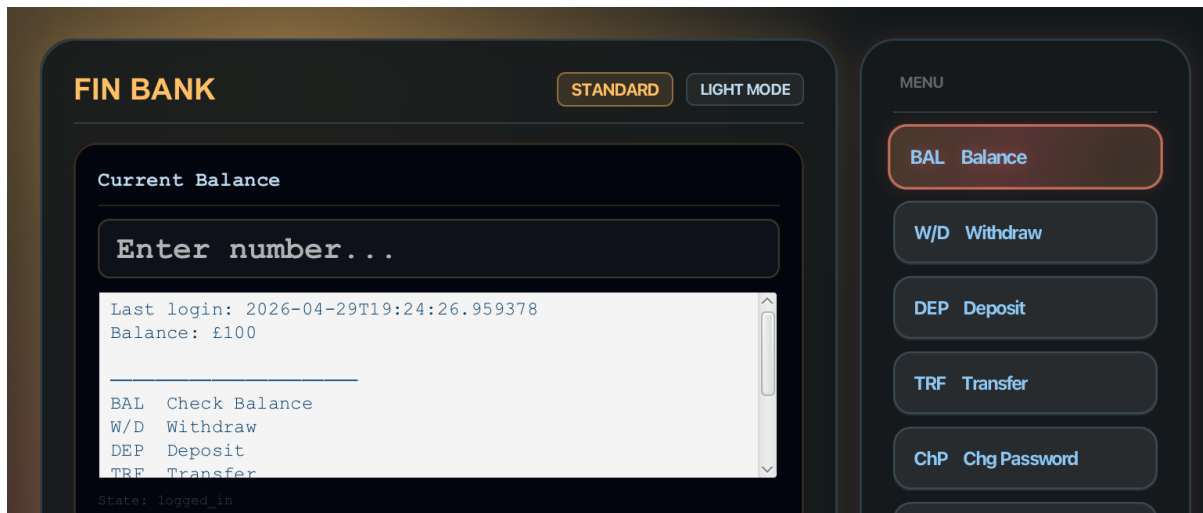


FIGURE: THE BALANCE SCREEN SHOWING ACCOUNT TYPE, CURRENT BALANCE, AND 'LAST LOGIN

4.3.7 Auto-Save and Audit Log

DataManager serialises account data after every transaction so balances persist between sessions. AuditLogger writes a timestamped entry to audit.log for every action, providing the tamper-evident trail required by FCA regulations.

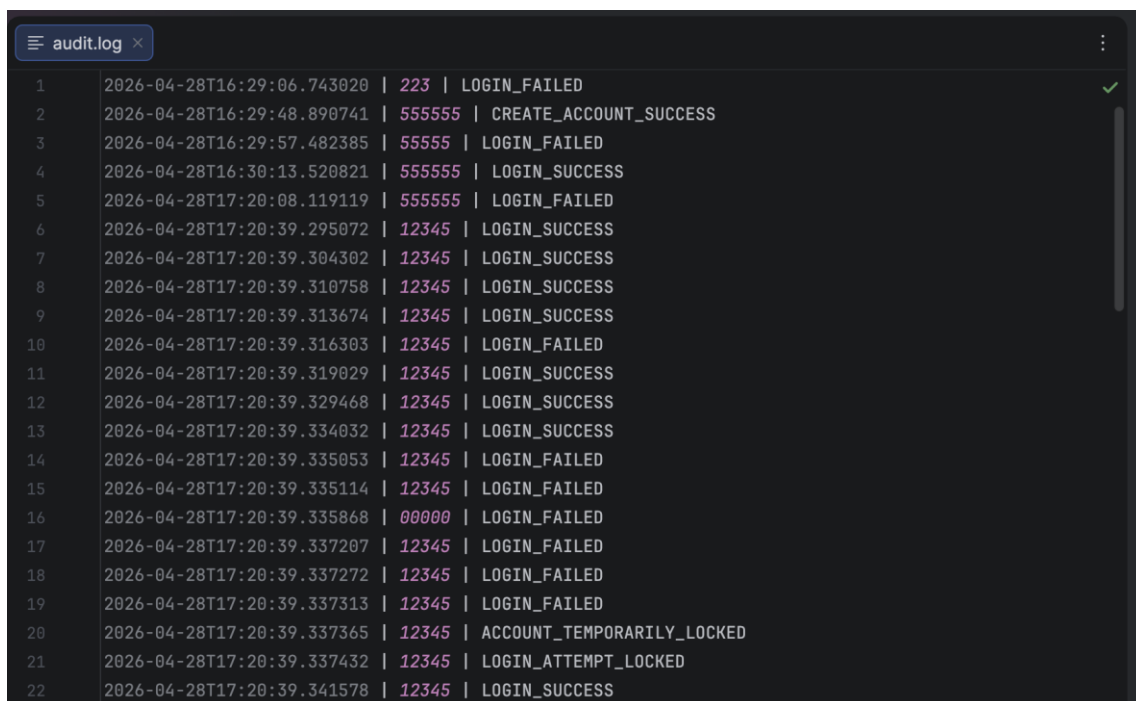


FIGURE: OPEN AUDIT.LOG IN INTELLIJ — THIS DEMONSTRATES THE AUDIT TRAIL WORKING.

4.3.8 Liquid Glass UI Design

The View was rewritten with an animated mesh gradient background, semi-transparent glass panels, holographic button hover effects, and a light/dark mode toggle with cross-fade transitions.

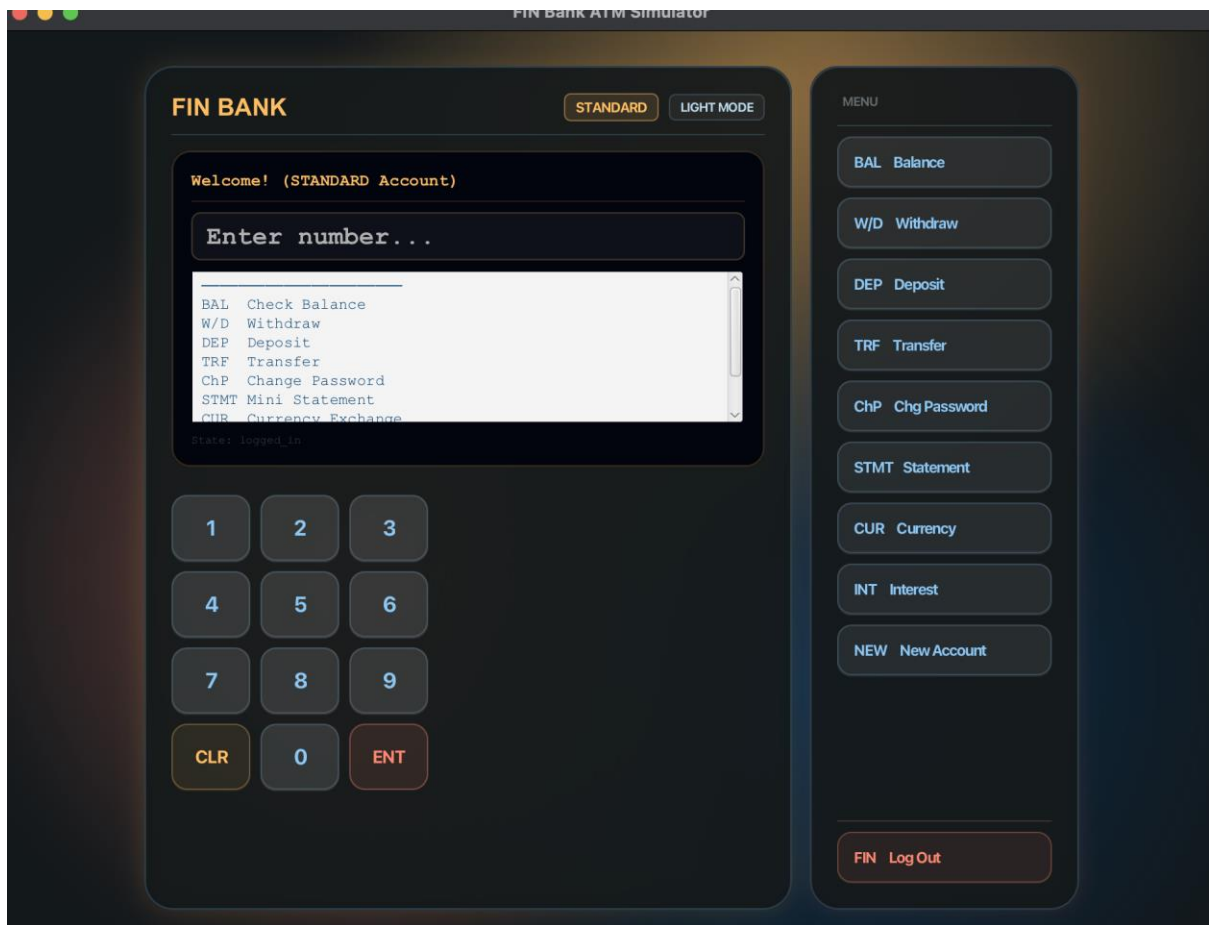


FIGURE: DARK MODE MAIN SCREEN LOGGED IN — SHOW THE GLASS PANELS, ANIMATED BLOBS, AND ACCOUNT TYPE BADGE

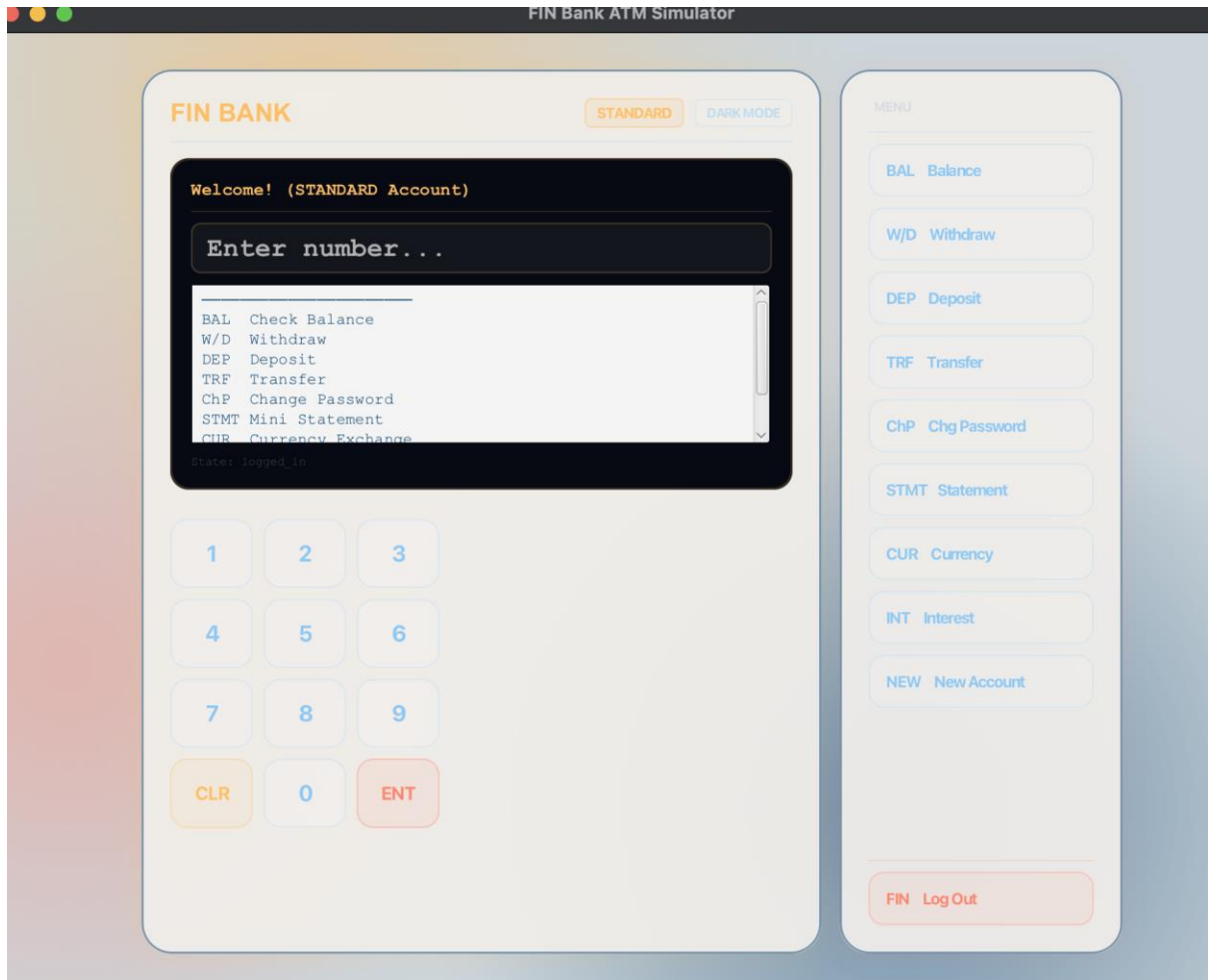


FIGURE: LIGHT MODE — SHOW THE SAME SCREEN AFTER CLICKING 'LIGHT MODE' TO DEMONSTRATE THE TOGGLE

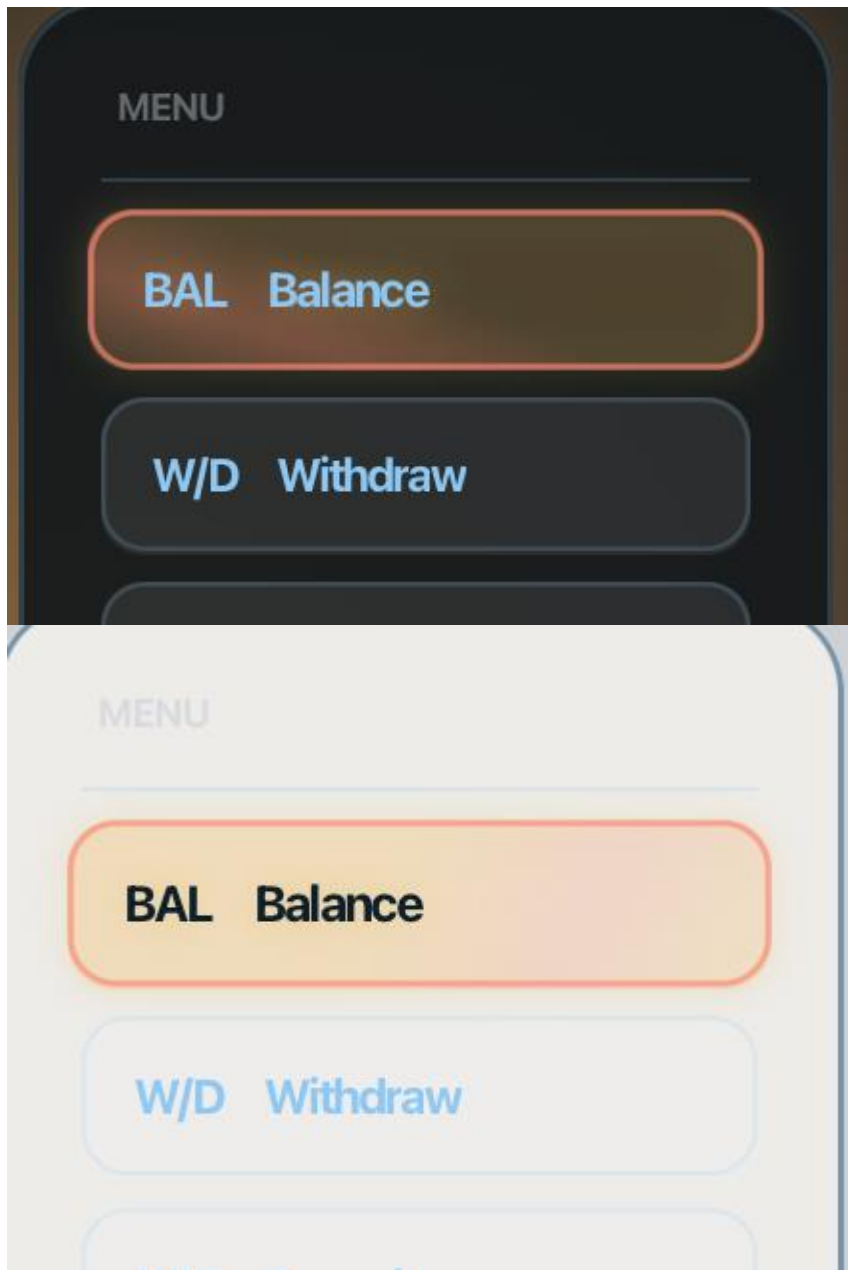


FIGURE: HOVER STATE ON A MENU BUTTON SHOWING THE HOLOGRAPHIC SHIMMER EFFECT — SCREENSHOT WHILE HOVERING OVER THE BAL BUTTON.

4.4 Automated Testing — JUnit 5

To verify the correctness of our business logic and to protect against regressions during development, we implemented an automated test suite using JUnit 5. Testing was performed by Amr and Haaroun. The test file `BankTest.java` is located in `src/test/java/com/atmbanksimulator/` and contains 58 tests.

4.4.1 What We Tested

- **BankAccount:** deposit positive amount succeeds; deposit negative amount fails; withdrawal within balance succeeds; withdrawal exceeding balance fails; balance correctly updated after both operations.
- **StudentAccount:** withdrawal within daily limit succeeds; cumulative withdrawals exceeding daily limit are blocked; `getRemainingLimit()` returns correct value; `resetDailyLimit()` resets to zero.
- **SavingAccount:** `addInterest()` correctly increases balance by 2%; interest amount is calculated correctly; zero-balance account does not gain interest.
- **Bank — login security:** correct credentials succeed; wrong password fails and increments attempt counter; third failure triggers timed lock; locked account rejects further attempts; correct login resets attempt counter.
- **Bank — transfer:** successful transfer correctly debits source and credits target; transfer to non-existent account fails; transfer to self fails; insufficient funds transfer fails.
- **Transaction:** transaction records correct type, amount, and balance; `getMiniStatement()` returns only the last 5 entries; masked account number appears correctly in output.
- **PrimeAccount — overdraft:** tests were written to verify that withdrawals exceeding the balance but within the £100 overdraft limit would produce a correct negative balance. All automated tests passed; however, during manual testing the overdraft feature was found to behave incorrectly, with the balance not reflecting the expected negative value after an overdraft withdrawal. The underlying bug in `handleOverdraft()` remains unresolved in the final submission as noted in Section 7.2, and highlights a limitation of the test suite — the written tests did not sufficiently cover the edge cases needed to catch this behaviour automatically.

4.4.2 Example Test Cases

The following examples illustrate the style and structure of the test suite:

```
@Test
void depositPositiveAmountShouldSucceed() {
    BankAccount acc = new BankAccount("10001", "11111", 100);
    assertTrue(acc.deposit(50));
    assertEquals(150, acc.getBalance());
}

@Test
void studentAccountShouldBlockOverDailyLimit() {
    StudentAccount acc = new StudentAccount("10002", "22222", 200);
    assertTrue(acc.withdraw(40)); // within £50 limit
    assertFalse(acc.withdraw(20)); // would exceed limit
}

@Test
void thirdFailedLoginShouldLockAccount() {
    Bank bank = new Bank();
    bank.addBankAccount("10001", "11111", 100, Bank.AccountType.STANDARD);
    bank.login("10001", "wrong");
    bank.login("10001", "wrong");
    bank.login("10001", "wrong");
    assertTrue(bank.isAccountLocked("10001"));
}
```

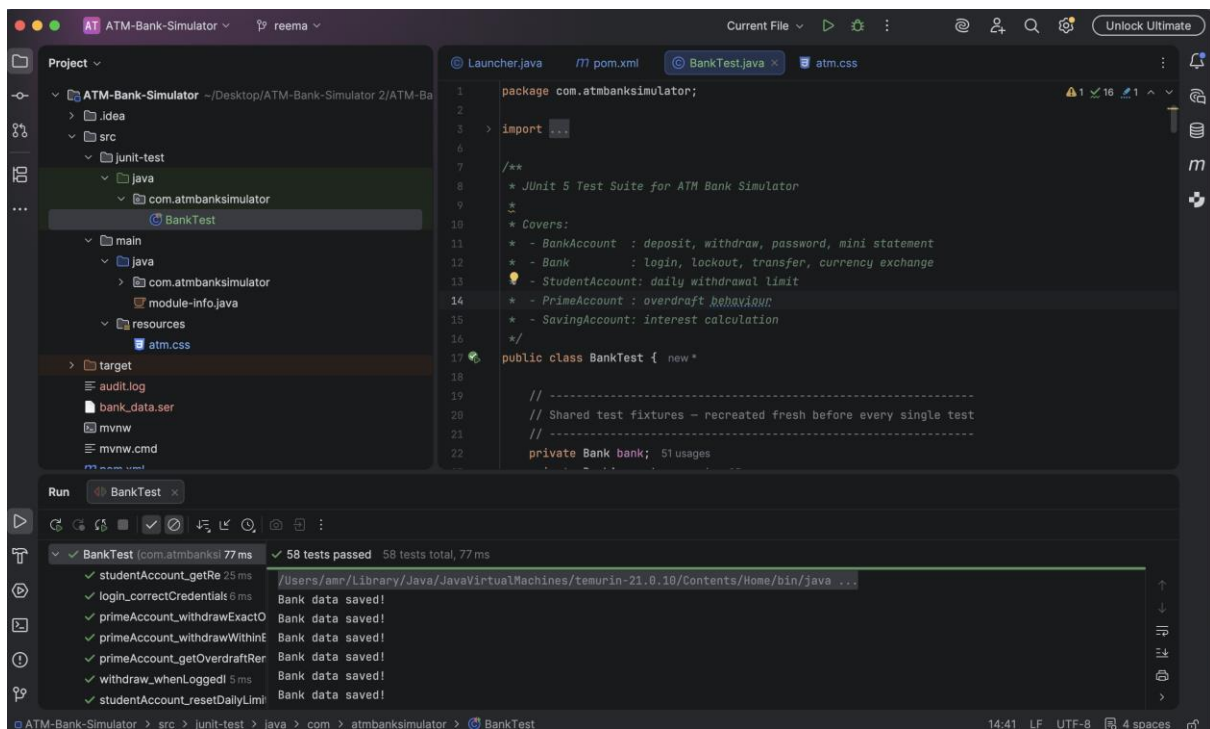


FIGURE: SCREENSHOT OF RUN THE TEST SUITE IN INTELLIJ AND GREEN 'ALL TESTS PASSED' RESULT PANEL.

5. Legal and Ethical Issues

5.1 Data Protection and GDPR

Any real-world deployment would be subject to UK GDPR and the Data Protection Act 2018. Account numbers, PINs, transaction histories, and balances constitute personal financial data requiring secure storage and restricted access. Our AuditLogger creates a record of all data access, and account numbers are masked in the Mini Statement. Passwords are currently stored as plaintext — a known limitation requiring bcrypt hashing before real deployment — and DataManager would need replacing with encrypted database storage.

5.2 Computer Misuse Act 1990

The Computer Misuse Act 1990 makes unauthorised access to computer systems a criminal offence (UK Government, 1990). Our login lockout feature is an important security measure: after three consecutive failed PIN attempts the account locks for five minutes, preventing the kind of systematic brute-force attack that would constitute an attempted offence under this Act. Our AuditLogger records every login attempt with a timestamp, creating a permanent trail as evidence in the event of a security incident.

5.3 Ethical Considerations

Our four account types apply rules fairly through a consistent polymorphic withdraw() implementation — the same logic applies to every user of each type with no hidden exceptions. The timed lock is a deliberate ethical choice: a permanent lock could strand a legitimate user without access to their funds. The large transaction confirmation reduces accidental or coerced high-value withdrawals, and the currency exchange feature displays indicative rates with a clear disclaimer to avoid misleading users.

5.4 Accessibility

The application does not yet implement screen reader support, keyboard-only navigation beyond the welcome screen ENTER key, or configurable font sizes. Under the UK Equality Act 2010 and WCAG 2.1, financial services must be accessible to all users including those with visual or motor impairments. The light/dark mode toggle and consistent colour contrast are initial steps, but a production system would need JavaFX Accessible API labels and full keyboard navigation.

5.5 Social Considerations

Beyond legal compliance, ATM software design carries real social responsibility. In the UK, approximately 1.1 million adults remain unbanked or reliant on cash, making ATMs a critical access point for economically vulnerable people (Financial Conduct Authority, 2022).

Complex flows, small text, or the absence of audio guidance can exclude elderly, disabled, or digitally limited users — our multi-step flows were designed with clarity in mind, but production deployment would require user testing with diverse groups. Account type differentiation also carries a social dimension: rules must be applied transparently to avoid discriminatory outcomes, which our polymorphic implementation ensures (Kamuangu, 2024).

6. Research

6.1 What Banks Look For in ATM Software Providers

As part of our project research, we investigated what commercial banks and financial institutions require when selecting ATM software vendors — to understand how our simulator compares to industry-grade solutions and what gaps exist. This research was led by Amr.

Banks evaluate ATM software vendors against several core criteria (Retail Banking Research, 2023):

- **Security Compliance:** PCI DSS certification is mandatory. Vendors must demonstrate HSM (Hardware Security Module) integration for PIN encryption, end-to-end encryption of all card data, and documented vulnerability management processes. Our simulator meets none of these production-grade requirements, though its layered security features (timed lock, audit log, PIN masking) reflect an understanding of the principles.
- **Reliability and Uptime:** Banks require 99.9%+ uptime SLAs (Service Level Agreements) for ATM software. This means software must handle edge cases gracefully — our boolean-return design on all financial operations (rather than throwing exceptions) is an example of defensive coding that contributes to reliability.
- **Interoperability:** Commercial ATM software must integrate with card networks (Visa, Mastercard), core banking systems (e.g. Temenos, Finastra), and national payment schemes (UK Faster Payments, BACS). This typically requires implementation of the ISO 8583 financial transaction message standard. Our simulator operates in isolation but its MVC architecture would make adding an API integration layer straightforward.
- **Auditability:** Regulators require complete audit trails of all ATM events. Our `AuditLogger.java` is a direct implementation of this principle — logging every login, transaction, and error with a timestamp. Production audit logs would additionally be signed to detect tampering and replicated to a remote server.
- **Accessibility:** The UK Equality Act requires ATMs to be accessible to users with disabilities. This includes audio guidance, large-print modes, and tactile keypads. Our light/dark mode toggle and consistent colour contrast are small steps toward accessibility, but a production system would need significantly more.

6.2 Computing Career Opportunities in FinTech

FinTech provides a wide range of career opportunities in computing, including software engineer, cybersecurity analyst, data scientist, compliance officer, and product manager (Kamuangu, 2024). Skills in languages such as Java and Python, alongside knowledge of cloud platforms and financial regulations, are highly valued by employers including Revolut, Monzo, and Stripe. Some other roles are:

- **Software Engineer / Java Developer:** Core transaction system development. Java remains dominant in banking due to its platform independence and mature ecosystem. Our project demonstrates OOP proficiency, MVC architecture, and JavaFX — directly applicable skills.
- **Security Engineer:** Responsible for authentication systems, encryption, and penetration testing. Our security features (login lock, audit log) reflect this domain.
- **Data Engineer / Data Scientist:** Builds pipelines for transaction stream processing and fraud detection models. Python, Apache Spark, and SQL are core skills.
- **UX/UI Designer:** Designs usable, accessible financial interfaces. Our Liquid Glass redesign demonstrates awareness of both aesthetics and usability.
- **DevOps / Cloud Engineer:** Maintains CI/CD pipelines and cloud-hosted financial infrastructure (AWS, Azure, GCP).

6.3 Future Trends in FinTech

- **Artificial Intelligence:** Real-time fraud detection, robo-advisors, and AI-powered credit scoring. McKinsey (2023) reports that over 60% of large financial institutions are now deploying AI in production risk systems. Our AuditLogger — which records every action with a timestamp — is an early version of the kind of structured event stream that AI fraud-detection systems consume.
- **Regulated APIs** allow authorised third parties to access customer financial data with consent. Our MVC architecture, which separates the Bank domain from the View, would make adding a REST API layer straightforward — the Bank class would simply be called by an API controller rather than a JavaFX UIModel.
- **Central Bank Digital Currencies (CBDCs):** Over 130 countries are currently exploring digital currency programmes. The Bank of England's digital pound project is in active

development. Our currency exchange feature demonstrates awareness of multi-currency systems, though real CBDC implementation would require cryptographic token management far beyond the scope of this prototype.

- **Post-Quantum Cryptography:** Current cryptographic standards (RSA, ECC) are theoretically vulnerable to quantum computers. Financial institutions are beginning to plan migration to post-quantum algorithms — a trend that will create significant demand for security-aware software engineers.

7. Critical Review

7.1 Three Strengths

Strength 1 — Genuine Feature Depth: The application goes significantly beyond the prototype brief. Features like auto-save with DataManager, the audit log, timed account lock, large transaction confirmation, and dynamic account creation are all found in real ATM software. The team's decision to implement these — rather than stopping at the minimum requirement — produced a demonstrably professional FinTech prototype.

Strength 2 — Clear MVC Separation: The View never calls Bank directly. This separation meant the complete UI rewrite was possible without touching any business logic — a direct demonstration of why architectural discipline matters in software development. Adding the Create Account feature required adding one UIModel state and one Bank method; the View simply needed a new button.

Strength 3 — Automated Test Suite: The JUnit 5 test suite provides formal verification that the core business logic behaves correctly across 58 tests, covering deposits, withdrawals, transfers, login security, daily limits, and interest calculations. The suite successfully caught and verified the majority of business logic, and where it fell short — as with the PrimeAccount overdraft behaviour, which passed automated tests but failed during manual testing — it demonstrated a valuable real-world lesson: automated testing is only as effective as the edge cases written into it. This kind of critical reflection on test coverage is itself a mark of professional software development practice.

7.2 Four Things We Would Change

Improvement 1 — PIN Hashing: Passwords are currently stored and compared as plain strings. We would implement bcrypt hashing (using the jBCrypt library) so that even if the serialised data file is accessed, PINs cannot be recovered. This is a fundamental security requirement for any real deployment.

Improvement 2 — Real Persistence (Database): DataManager uses Java object serialisation, which is fragile and not interoperable. We would replace it with an SQLite

database via JDBC. This would allow transaction history to be queried, accounts to be searched, and audit logs to be stored in a proper relational structure that tools like Excel or reporting software can read.

Improvement 3 — Expanded Test Coverage and UI Tests: Our JUnit suite covers business logic but not the View or Controller. We would add TestFX tests (a JavaFX UI testing library) to verify that pressing the W/D button while in the correct state correctly invokes `processWithdraw()`, and that the password masking works as expected at the View level. Full coverage would give us confidence to refactor any part of the codebase without breaking existing behaviour.

Improvement 4 – Fully implementing the Overdraft feature. `PrimeAccount` was designed to allow withdrawals up to £100 beyond the account balance, producing a correct negative balance. A fix was attempted using `setBalance()` to directly assign the correct negative value after the standard `withdraw()` failed, however the feature still did not behave correctly during manual testing. Given more time, we would have used the JUnit tests written for this feature to systematically step through the `handleOverdraft()` call chain and identify where the value was being lost. The architecture and intent are correct — it was a debugging time constraint rather than a fundamental design flaw.

8. Statement of Individual Contribution - *Written individually*

8.1 Daniella — Lead Developer (Student No. 25817373)

I led the technical development of the project, coordinating code integration across all branches and resolving merge conflicts. My direct code contributions were:

PrimeAccount.java; SavingAccount.java (2% interest calculation and preview); the transfer() method in Bank.java (two-step withdraw/deposit with self-transfer protection); co-authorship of View.java (animated mesh gradient using AnimationTimer, holographic button Timeline effect). I reviewed pull requests from all team members and was the final approver before any code was merged to main. In addition to development, I was responsible for the majority of the report writing and formatting, covering Sections 1, 2, 3, 4.1–4.3, 5, 7, and 8, as well as the overall document structure. I led the structure and content for the presentation.

8.2 Reema — Project Manager, Core Developer (Student No. 25838268)

I managed the project timeline, organised sprint goals, and assigned tasks. I facilitated team coordination through weekly progress reviews. My code contributions were: the Create Account feature (full UIModel state + Bank method implementation, dynamic user registration without hardcoded accounts); the 5-minute Temporary Lock system (timestamp-based timed unlock); the Last Attempt Warning message displayed on the second failed login attempt; AuditLogger.java (writing all actions to audit.log with timestamps); DataManager.java (serialising and loading account data for auto-save after every transaction and on startup); the Large Transaction Confirmation flow (second ENT press for withdrawals over £500, preventing accidental high-value cashouts); masked account numbers (****) in the Mini Statement with integrated last-5-transaction display; and Last Login Time display on the Balance screen. I handled branch integration, resolved merge conflicts, and ensured the final codebase remained stable before submission.

8.3 Haaroun — Security and QA (Student No. 25811005)

I designed and implemented the UIModel finite-state machine, expanding it from the original 4 states to 9 states to support all new features. I implemented: processTransfer() (two-step flow), processMiniStmt(), processCurrency() (two-step currency selection), processInterest() (SavingAccount type check and preview). I built the login attempt tracking system in Bank.java (loginAttempts HashMap, lockedAccounts HashSet, getRemainingLoginAttempts(), isAccountLocked()). I co-wrote the JUnit 5 test suite with

Amr, responsible for the Bank login security tests and the StudentAccount daily limit tests. I contributed to the formatting and layout of the presentation.

8.4 Amr — Research and UI (Student No. 25800030)

I conducted the business and market research for Section 6, investigating what commercial banks require from ATM software vendors, analysing industry standards (PCI DSS, ISO 8583, WCAG), and reviewing future FinTech trends. For the UI, I implemented the light/dark mode toggle with cross-fade transitions (ensuring all text remains readable in both themes) and designed and built the welcome screen and goodbye screen overlays with the 5-second PauseTransition sequencing. I co-wrote the JUnit 5 test suite with Haaroun, and wrote section 4.4 of the report on the testing. Contributed to the formatting and layout of the presentation.

Word Count: 1758 words

9. References

Arner, D.W., Barberis, J. and Buckley, R.P. (2015) 'The evolution of Fintech: A new post-crisis paradigm?', *Georgetown Journal of International Law*, 47(4), pp. 1271–1319.

Bank for International Settlements (2023) *CBDCs: An opportunity for the monetary system* [Online]. Available at: <https://www.bis.org/publ/arpdf/ar2021e3.htm> (Accessed: 10 April 2026).

Financial Conduct Authority (2022) *Our approach to FinTech* [Online]. Available at: <https://www.fca.org.uk/firms/innovation/our-approach-fintech> (Accessed: 10 April 2026).

Information Commissioner's Office (2023) *Guide to UK GDPR* [Online]. Available at: <https://ico.org.uk/for-organisations/uk-gdpr-guidance-and-resources/> (Accessed: 10 April 2026).

InnReg (2024) *Top 6 fintech trends to watch in 2024* [Online]. Available at: <https://www.innreg.com/blog/fintech-trends-2024> (Accessed: 28 April 2026).

Kamuangu, P. (2024) 'A Review on Cybersecurity in Fintech', *Journal of Economics, Finance and Accounting Studies*, 6(1), pp. 47–53.

McKinsey & Company (2023) *The state of AI in 2023: Generative AI's breakout year* [Online]. Available at: <https://www.mckinsey.com/capabilities/quantumblack/our-insights/the-state-of-ai-in-2023> (Accessed: 10 April 2026).

Mermaid (2024) *Mermaid Live Editor* [Online]. Available at: <https://mermaid.live> (Accessed: 29 April 2026).

Nasir, A. et al. (2023) 'Cybersecurity threats in FinTech: A systematic review', *Expert Systems with Applications*, 241.

Oracle Corporation (2023) *JavaFX Documentation* [Online]. Available at: <https://openjfx.io/javadoc/17/> (Accessed: 28 April 2026).

Payment Card Industry Security Standards Council (2022) *PCI Data Security Standard (PCI DSS) version 4.0* [Online]. Available at: <https://www.pcisecuritystandards.org> (Accessed: 12 April 2026).

Retail Banking Research (2023) *Global ATM Market and Forecasts*. London: RBR.

UK Government (1990) *Computer Misuse Act 1990*. London: HMSO.

UK Government (2018) *Data Protection Act 2018*. London: HMSO.

World Wide Web Consortium (2018) *Web Content Accessibility Guidelines (WCAG) 2.1* [Online]. Available at: <https://www.w3.org/TR/WCAG21/> (Accessed: 28 April 2026).

10. Appendices

Appendix A — Test Account Credentials

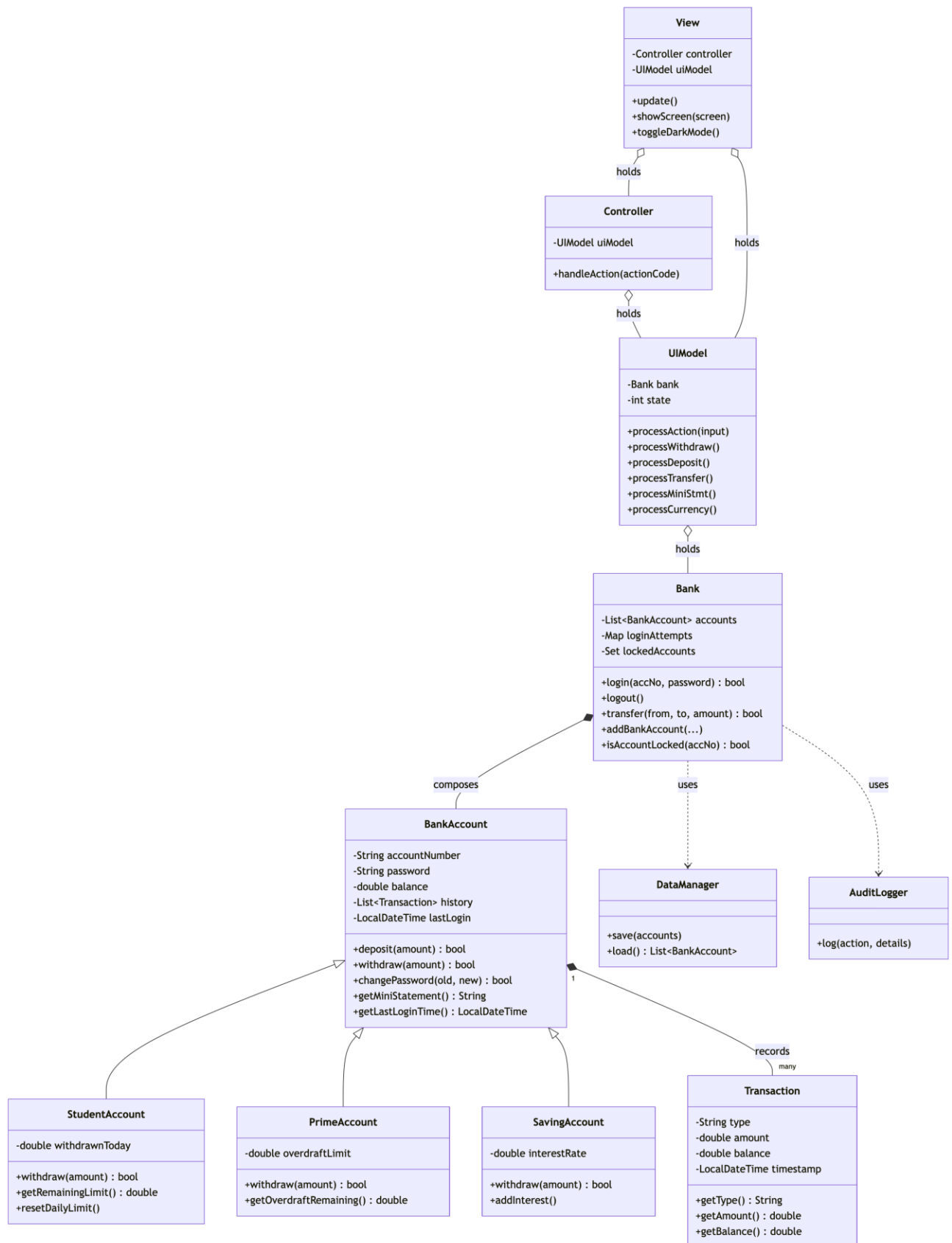
Account No.	Password	Balance	Type	Feature
10001	11111	£100	STANDARD	Standard account
10002	22222	£50	STUDENT	£50 daily withdrawal limit
10003	33333	£1,000	PRIME	Investor account for large deposits
10004	44444	£500	SAVING	2% interest — press INT
10005	55555	£200	STANDARD	Transfer target for demos
10006	66666	£750	PRIME	Second Prime for overdraft demos

Appendix B — GitHub Repository

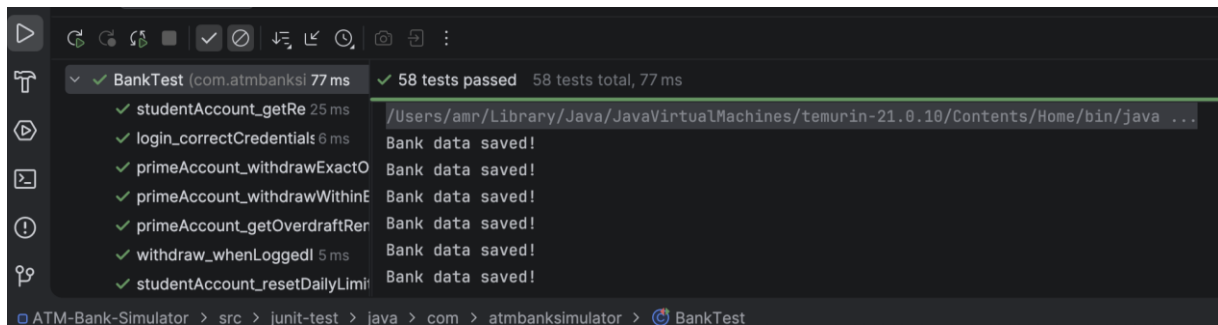
Source code: <https://github.com/lellacf26658-dev/ATM-Bank-Simulator-2.0.git>

Demonstration Video: [Final finbank atm demo.mov](#)

Appendix C — Class Diagram



Appendix D — JUnit Test Results



Appendix E — Sample audit.log Output

```
[2026-05-01 14:32:11] LOGIN_ATTEMPT account=10001 result=FAILED
attempts=1
[2026-05-01 14:32:19] LOGIN_ATTEMPT account=10001 result=FAILED
attempts=2
[2026-05-01 14:32:21] LOGIN_ATTEMPT account=10001 result=FAILED
attempts=3
[2026-05-01 14:32:21] ACCOUNT_LOCKED account=10001 duration=5min
[2026-05-01 14:40:05] LOGIN_SUCCESS account=10003 type=PRIME
[2026-05-01 14:41:03] MINI_STATEMENT account=10003 requested
[2026-05-01 14:41:45] LOGOUT account=10003
```

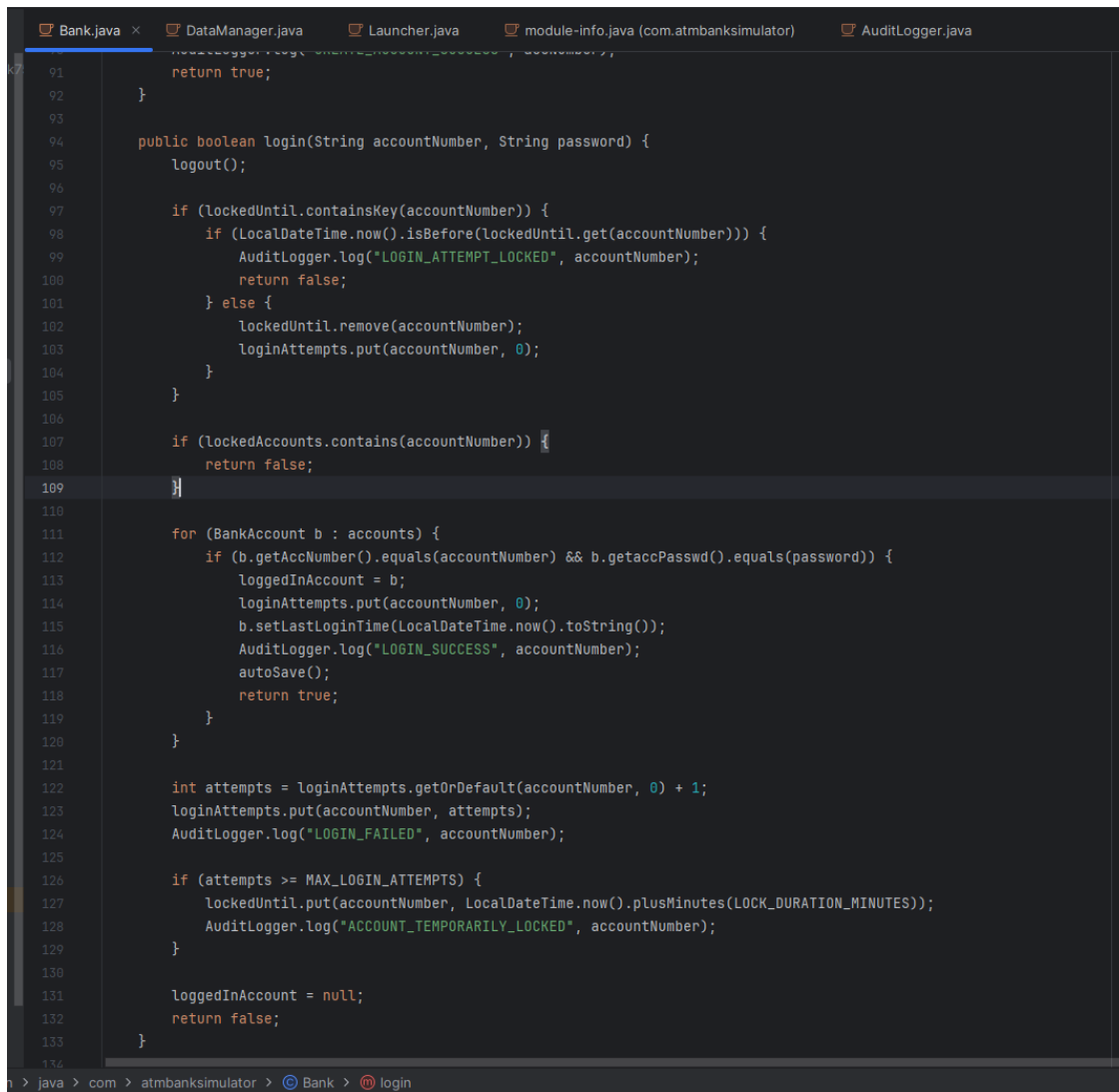
Appendix F — Key Code Implementations

The following screenshots illustrate the key additions and modifications made to the original prototype codebase.

AuditLogger.java — logs every sensitive action (login attempts, account creation, locks) to audit.log with a timestamp. Added by Reema; directly implements FCA audit trail requirements.



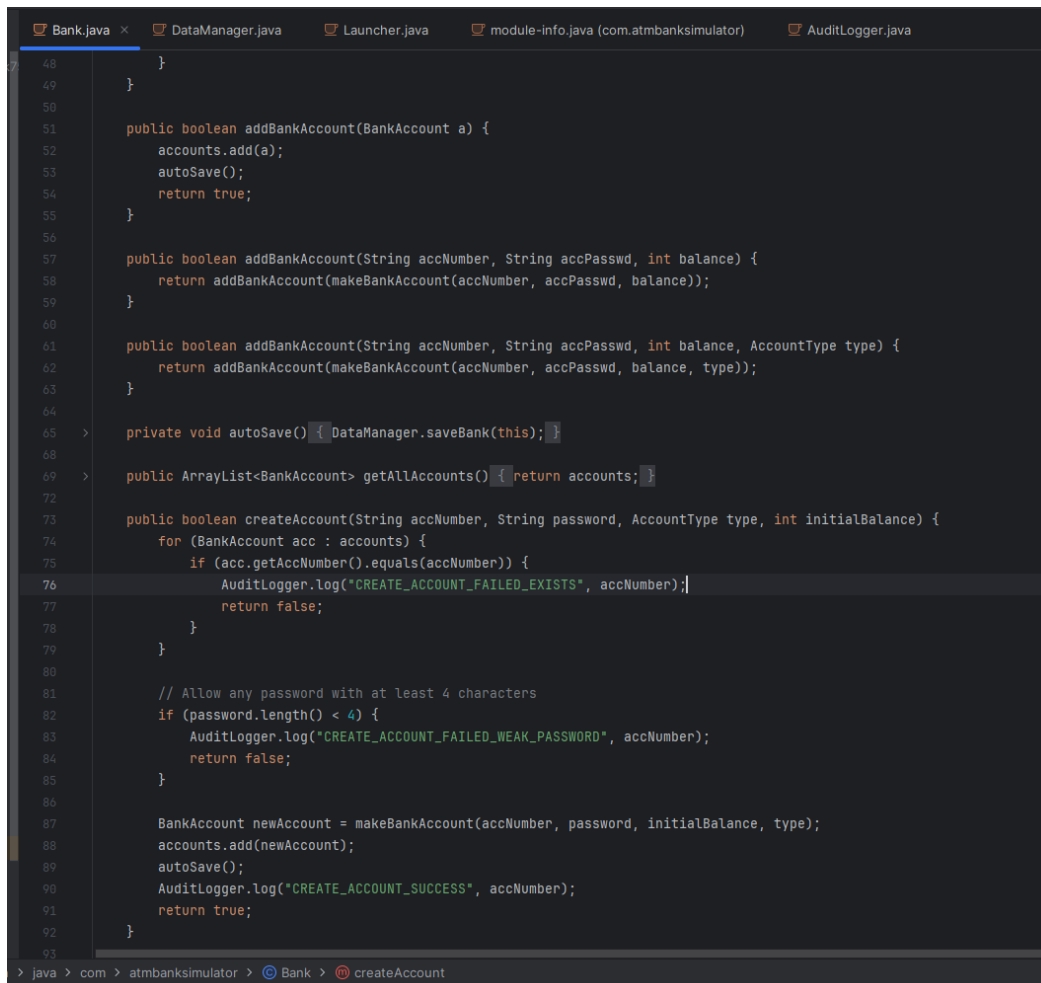
Bank.java login() — shows the three-attempt timed lock: failed logins increment a counter; on the third failure, lockedUntil is set to now + 5 minutes. Implemented by Reema and Haaroun.



```
Bank.java x DataManager.java Launcher.java module-info.java (com.atmbanksimulator) AuditLogger.java
91         return true;
92     }
93
94     public boolean login(String accountNumber, String password) {
95         logout();
96
97         if (lockedUntil.containsKey(accountNumber)) {
98             if (LocalDateTime.now().isBefore(lockedUntil.get(accountNumber))) {
99                 AuditLogger.log("LOGIN_ATTEMPT_LOCKED", accountNumber);
100                 return false;
101             } else {
102                 lockedUntil.remove(accountNumber);
103                 loginAttempts.put(accountNumber, 0);
104             }
105         }
106
107         if (lockedAccounts.contains(accountNumber)) {
108             return false;
109         }
110
111         for (BankAccount b : accounts) {
112             if (b.getAccNumber().equals(accountNumber) && b.getaccPasswd().equals(password)) {
113                 loggedInAccount = b;
114                 loginAttempts.put(accountNumber, 0);
115                 b.setLastLoginTime(LocalDateTime.now().toString());
116                 AuditLogger.log("LOGIN_SUCCESS", accountNumber);
117                 autoSave();
118                 return true;
119             }
120         }
121
122         int attempts = loginAttempts.getOrDefault(accountNumber, 0) + 1;
123         loginAttempts.put(accountNumber, attempts);
124         AuditLogger.log("LOGIN_FAILED", accountNumber);
125
126         if (attempts >= MAX_LOGIN_ATTEMPTS) {
127             lockedUntil.put(accountNumber, LocalDateTime.now().plusMinutes(LOCK_DURATION_MINUTES));
128             AuditLogger.log("ACCOUNT_TEMPORARILY_LOCKED", accountNumber);
129         }
130
131         loggedInAccount = null;
132         return false;
133     }
134 }
```

java > com > atmbanksimulator > Bank > login

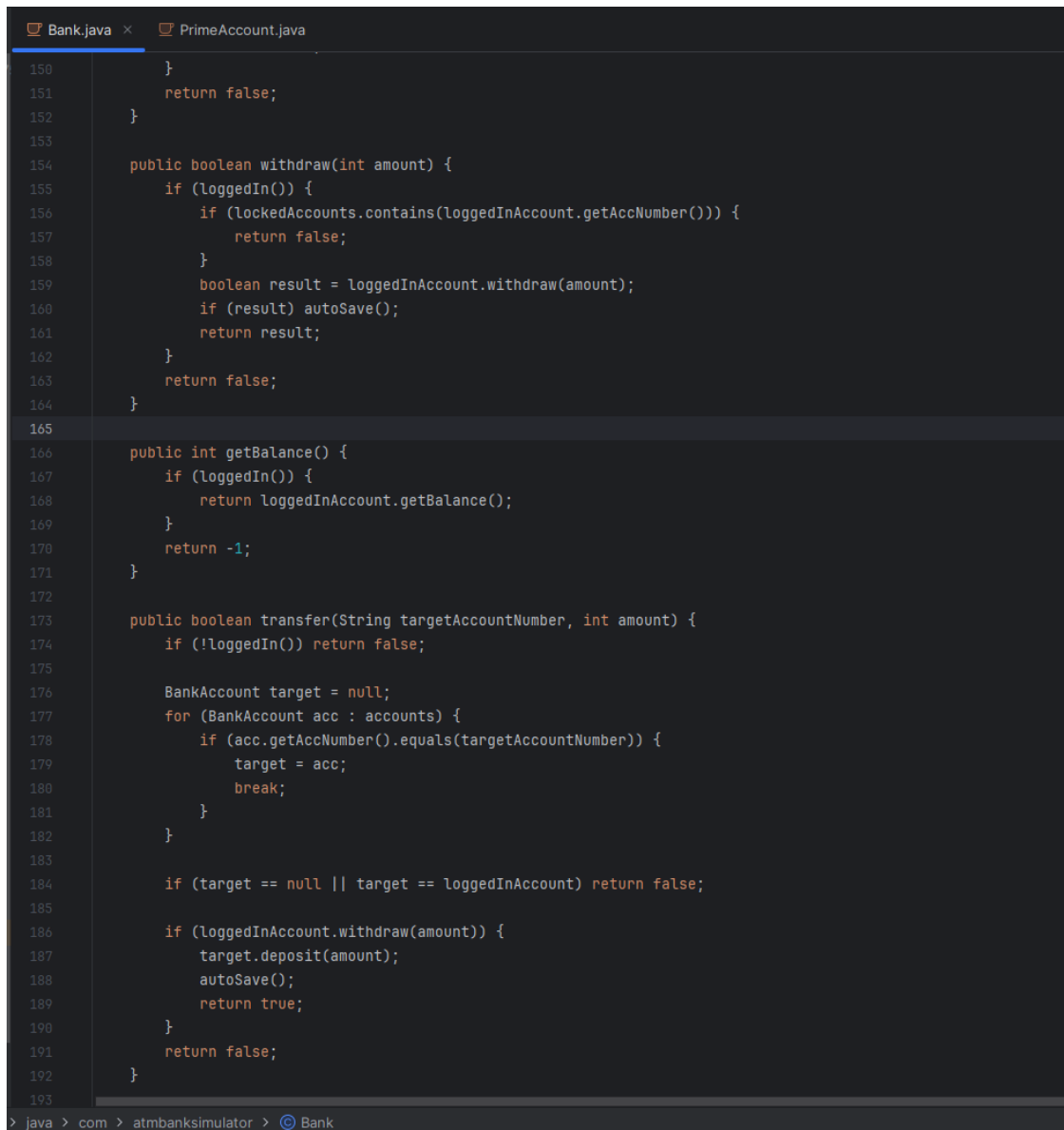
Bank.java createAccount() — validates uniqueness and minimum password length before adding the new account to the ArrayList and auto-saving. Added by Reema.



```
48     }
49 }
50
51 public boolean addBankAccount(BankAccount a) {
52     accounts.add(a);
53     autoSave();
54     return true;
55 }
56
57 public boolean addBankAccount(String accNumber, String accPasswd, int balance) {
58     return addBankAccount(makeBankAccount(accNumber, accPasswd, balance));
59 }
60
61 public boolean addBankAccount(String accNumber, String accPasswd, int balance, AccountType type) {
62     return addBankAccount(makeBankAccount(accNumber, accPasswd, balance, type));
63 }
64
65 > private void autoSave() { DataManager.saveBank(this); }
66
67 > public ArrayList<BankAccount> getAllAccounts() { return accounts; }
68
69
70 public boolean createAccount(String accNumber, String password, AccountType type, int initialBalance) {
71     for (BankAccount acc : accounts) {
72         if (acc.getAccNumber().equals(accNumber)) {
73             AuditLogger.log("CREATE_ACCOUNT_FAILED_EXISTS", accNumber);
74             return false;
75         }
76     }
77
78     // Allow any password with at least 4 characters
79     if (password.length() < 4) {
80         AuditLogger.log("CREATE_ACCOUNT_FAILED_WEAK_PASSWORD", accNumber);
81         return false;
82     }
83
84     BankAccount newAccount = makeBankAccount(accNumber, password, initialBalance, type);
85     accounts.add(newAccount);
86     autoSave();
87     AuditLogger.log("CREATE_ACCOUNT_SUCCESS", accNumber);
88     return true;
89 }
90
91 }
92
93
```

java > com > atmbanksimulator > Bank > createAccount

Bank.java transfer() — two-step transfer: withdraw from source, deposit to target. Guards against self-transfer and non-existent accounts. Implemented by Daniella.



```
150     }
151     return false;
152 }
153
154 public boolean withdraw(int amount) {
155     if (loggedIn()) {
156         if (lockedAccounts.contains(loggedInAccount.getAccNumber())) {
157             return false;
158         }
159         boolean result = loggedInAccount.withdraw(amount);
160         if (result) autoSave();
161         return result;
162     }
163     return false;
164 }
165
166 public int getBalance() {
167     if (loggedIn()) {
168         return loggedInAccount.getBalance();
169     }
170     return -1;
171 }
172
173 public boolean transfer(String targetAccountNumber, int amount) {
174     if (!loggedIn()) return false;
175
176     BankAccount target = null;
177     for (BankAccount acc : accounts) {
178         if (acc.getAccNumber().equals(targetAccountNumber)) {
179             target = acc;
180             break;
181         }
182     }
183
184     if (target == null || target == loggedInAccount) return false;
185
186     if (loggedInAccount.withdraw(amount)) {
187         target.deposit(amount);
188         autoSave();
189         return true;
190     }
191     return false;
192 }
193
```

> java > com > atmbanksimulator > © Bank

PrimeAccount.java — overdraft handling. The original code calculated a deficit but never applied it; a fix was attempted using `setBalance()` to produce a negative balance within the £100 overdraft limit, however the overdraft feature remains non-functional in the final

submission. Given more time, the correct approach would be to ensure `handleOverdraft()` is reached only when the standard `withdraw()` has genuinely failed, and that `setBalance()` is called with the correct negative value before recording the transaction. Bug fix attempted by Daniella.

```

PrimeAccount.java x
37 // Constructor
38 // Calls the parent BankAccount constructor to set accNumber, accPasswd, balance.
39 // @param accNumber account number
40 // @param accPasswd account password
41 // @param balance starting balance
42 public PrimeAccount(String accNumber, String accPasswd, int balance) { super(accNumber, accPasswd, balance); }
43
44
45 // withdraw() - Overridden from BankAccount
46 //
47 // Normal BankAccount: cannot withdraw more than balance.
48 // PrimeAccount: can withdraw up to balance + overdraftLimit.
49 //
50 // Example: balance = £50, overdraftLimit = £100
51 // withdraw(£120) + allowed (50 + 100 = 150 >= 120)
52 // withdraw(£160) + rejected (exceeds 150)
53 // After withdraw(£120) → balance = -£70
54
55 @Override
56 public boolean withdraw(int amount) {
57 // Reject negative withdrawal amounts immediately
58 if (amount < 0) {
59 return false;
60 }
61
62 // Calculate the total funds available including overdraft headroom
63 // e.g. balance = £50, overdraftLimit = £100 → availableFunds = £150
64 int availableFunds = getBalance() + overdraftLimit;
65
66 if (amount > availableFunds) {
67 // Withdrawal exceeds the overdraft allowance - refuse it
68 return false;
69 }
70
71 // Attempt a standard withdrawal first (works if amount ≤ balance)
72 // If that succeeds, we're done - no overdraft needed.
73 // If it fails (balance < amount), fall through to handleOverdraft.
74 return super.withdraw(amount) || handleOverdraft(amount);
75 }
76
77 // handleOverdraft()
78 // Called when the normal withdraw() fails (i.e. amount > current balance).
79 // This method correctly sets the balance to a negative value within
80 // the allowed overdraft range.
81
82
83 // [BUG FIX EXPLANATION]
84 // ORIGINAL CODE:
85 // int deficit = amount - currentBalance; // + calculated but NEVER USED
86 // if (currentBalance > 0) {
87 //     super.withdraw(currentBalance); // + drains balance to 0
88 // }
89 // return true; // + balance stays at 0, NOT negative!
90 //
91 // The bug: the 'deficit' variable was computed but the remaining amount
92 // was never subtracted. The account always ended at £0 instead of going
93 // into a proper negative balance.
94 //
95 // FIX: Use setBalance() (inherited from BankAccount, protected access)
96 // to directly set balance = currentBalance - amount.
97 // This correctly produces a negative balance for overdraft scenarios.
98 // Then record the transaction with the correct overdraft type.
99 //
100 // @param amount - the withdrawal amount that the normal withdraw() rejected
101 // @return true if overdraft withdrawal is within the allowed limit
102 private boolean handleOverdraft(int amount) {
103 int currentBalance = getBalance();
104
105 // Double-check the overdraft range (should always be true here,
106 // but guarding against edge cases)
107 if (currentBalance - amount >= -overdraftLimit) {
108
109 // [FIX] Directly set the balance to the correct negative value.
110 // setBalance() is the protected setter in BankAccount that lets
111 // subclasses update the balance when normal deposit/withdraw won't do.
112 setBalance(currentBalance - amount); // e.g. 50 - 120 = -70 ✓
113
114 // Record this as a WITHDRAWAL transaction (not an overdraft fee,
115 // as fees are a future enhancement - see Daniella's Week 5+ task)
116 recordTransaction(Transaction.TransactionType.WITHDRAWAL, amount);
117
118 return true;
119 }
120
121 return false; // Should not reach here given the check in withdraw()
122 }
123
124

```

`SavingAccount.java` — `addInterest()` applies 2% interest and corrects the transaction type to `INTEREST` so it appears correctly in the Mini Statement. `getInterestPreview()` shows the user what they will earn before confirming. Implemented by Daniella.

```

PrimeAccount.java SavingAccount.java x
54 public void addInterest() {
55     int currentBalance = getBalance();
56
57     // Only apply interest if balance is positive (no interest on £0 or debt)
58     if (currentBalance <= 0) {
59         return; // Nothing to earn interest on
60     }
61
62     // Calculate interest amount (integer truncation = rounds down)
63     // e.g. £1000 * 0.02 = £20.0 → (int) = £20
64     int interest = (int) (currentBalance * interestRate);
65
66     if (interest <= 0) {
67         return; // Interest rounds to zero (very small balance) – skip
68     }
69
70     // Add the interest to the balance using the parent deposit() method.
71     // This also records a DEPOSIT transaction. We then overwrite that with
72     // an INTEREST transaction so the Mini Statement shows the correct type.
73     deposit(interest); // Deposit adds to balance and records DEPOSIT
74
75     // [NEW] Correct the transaction type: overwrite the DEPOSIT record
76     // with an INTEREST record so the Mini Statement reads correctly.
77     // recordTransaction() is protected – accessible from subclasses.
78     recordTransaction(Transaction.TransactionType.INTEREST, interest);
79
80     System.out.println("SavingAccount: Interest of £" + interest +
81         " applied. New balance: £" + getBalance());
82 }
83
84
85 // [NEW] getInterestPreview()
86 // Returns a formatted string showing how much interest the user would
87 // receive if they pressed the "Apply Interest" button right now.
88 // UIModel displays this when the user first presses "Int" so they
89 // know what will happen before it is applied.
90 public String getInterestPreview() {
91     int currentBalance = getBalance();
92
93     if (currentBalance <= 0) {
94         return "No interest applicable.\nBalance must be positive\nto earn interest.";
95     }
96
97     int interest = (int) (currentBalance * interestRate);
98
99     if (interest <= 0) {
100         return "Balance too low to earn\ measurable interest.\n(Rate: " +
101             (interestRate * 100) + "%)";
102     }
103 }

```

StudentAccount.java — cumulative daily limit enforcement: withdrawnToday tracks the session total and blocks any withdrawal that would exceed £50. getDailyLimitInfo() surfaces this to the user after each transaction. Implemented by Reema.

```

StudentAccount.java x
48 // On success, increments withdrawnToday.
49 @Override
50 public boolean withdraw(int amount) {
51     // Check daily limit - reject if it would be exceeded
52     if (withdrawnToday + amount > dailyLimit) {
53         return false;
54     }
55
56     // Attempt the normal withdrawal via parent class
57     boolean success = super.withdraw(amount);
58
59     // Only update the daily tracker if the withdrawal actually succeeded
60     if (success) {
61         withdrawnToday += amount;
62     }
63
64     return success;
65 }
66
67
68 // [NEW] getAccountTypeName()
69 // Overrides BankAccount's version to return "STUDENT".
70 @Override
71 public String getAccountTypeName() { return "STUDENT"; }
72
73
74
75
76 // getRemainingLimit() - Returns how much more can be withdrawn today
77 // [UNCHANGED from original]
78 public int getRemainingLimit() { return dailyLimit - withdrawnToday; }
79
80
81
82
83 // getWithdrawnToday() - Returns total withdrawn in current session
84 // [UNCHANGED from original]
85 public int getWithdrawnToday() { return withdrawnToday; }
86
87
88
89
90 // resetDailyLimit() - Resets the daily withdrawal counter to 0
91 // [UNCHANGED from original]
92 // Would be called at midnight in a real banking system.
93 public void resetDailyLimit() { withdrawnToday = 0; }
94
95
96
97
98 // [NEW] getDailyLimitInfo()
99 // Returns a formatted string showing the student their daily limit status.
100 // UIModel displays this after a StudentAccount withdrawal so the student
101 // always knows their remaining allowance.
102 public String getDailyLimitInfo() {
103     return String.format(
104         "Daily Limit:   %d\n" +
105         "Withdrawn:       %d\n" +
106         "Remaining:       %d",
107         dailyLimit, withdrawnToday, getRemainingLimit()
108     );
109 }

```

UIModel.java — the nine-state finite-state machine. The STATE_PASSWORD branch of processEnter() calls bank.login(), checks for a timed lock, and displays a last-attempt warning on the second failure. Designed and implemented by Haaroun

```

UIModel.java x
1 package com.atmbanksimulator;
2
3 public class UIModel {
4
5     View view;
6     private Bank bank;
7
8     private final String STATE_ACCOUNT_NO = "account_no";
9     private final String STATE_PASSWORD = "password";
10    private final String STATE_LOGGED_IN = "logged_in";
11    private final String STATE_CH_PASS = "change_password";
12    private final String STATE_TRANSFER_ACC = "transfer_acc";
13    private final String STATE_TRANSFER_AMT = "transfer_amt";
14    private final String STATE_MINI_STMT = "mini_stmt";
15    private final String STATE_CURRENCY = "currency_amt";
16    private final String STATE_CURRENCY_SEL = "currency_sel";
17    private final String STATE_INTEREST = "interest";
18    private final String STATE_CREATE_ACC = "create_account";
19
200
210    message = "Account: " + accNumber;
211    result = "Enter your PIN / password, then press Ent.";
212    }
213    } else if (state.equals(STATE_PASSWORD)) {
214        if (numberPadInput.isEmpty()) {
215            message = "Please enter your password.";
216        } else {
217            accPasswd = numberPadInput;
218            numberPadInput = "";
219            boolean success = bank.login(accNumber, accPasswd);
220            if (success) {
221                BankAccount acc = bank.getLoggedInAccount();
222                setState(STATE_LOGGED_IN);
223                message = "Welcome! (" + acc.getAccountTypeName() + " Account)";
224                result = buildLoggedInMenu();
225            } else {
226                if (bank.isAccountLocked(accNumber)) {
227                    setState(STATE_ACCOUNT_NO);
228                    message = "Account LOCKED for 5 minutes.";
229                    result = "Too many failed attempts.\nPlease try again later.";
230                } else {
231                    int remaining = bank.getRemainingLoginAttempts(accNumber);
232                    setState(STATE_ACCOUNT_NO);
233                    if (remaining == 1) {
234                        message = "WARNING: Last attempt before account LOCKED!";
235                        result = "Attempts remaining: " + remaining + "\nEnter account number:";
236                    } else {
237                        message = "Incorrect password.";
238                        result = "Attempts remaining: " + remaining + "\nEnter account number:";
239                    }
240                }
241            }
242        }
243    } else if (state.equals(STATE_LOGGED_IN)) {
244        if (!numberPadInput.isEmpty()) {
245            message = "Select an action first.";
246            result = buildLoggedInMenu();
247            numberPadInput = "";
248        } else {
249            message = "Select an option from the menu on the right.";
250            result = buildLoggedInMenu();
251        }
252    }
253    }
254    }

```

UIModel.java processWithdraw() — any withdrawal over £500 sets a confirmation flag and prompts the user to press W/D again. The transaction only proceeds on the second press. Added by Reema.


```

331     update();
332 }
333
334 public void processWithdraw() {
335     if (!bank.loggedIn()) { reset("Not logged in."); update(); return; }
336     if (numberPadInput.isEmpty()) {
337         message = "Enter amount, then W/D";
338         result = "Type the amount to withdraw then press W/D again.";
339         confirmLargeTransaction = false;
340     } else {
341         try {
342             int amount = Integer.parseInt(numberPadInput);
343             if (amount > 500 && !confirmLargeTransaction) {
344                 message = "Large withdrawal: £" + amount;
345                 result = "Press W/D again to confirm";
346                 confirmLargeTransaction = true;
347                 numberPadInput = "";
348                 update();
349                 return;
350             }
351             confirmLargeTransaction = false;
352             numberPadInput = "";
353             boolean success = bank.withdraw(amount);
354             if (success) {
355                 message = "Withdrawn: £" + amount;
356                 result = "New balance: £" + bank.getBalance() + "\n\n" + buildLoggedInMenu();
357             } else {
358                 message = "Withdrawal failed.";
359                 result = "Insufficient funds.\nBalance: £" + bank.getBalance() + "\n\n" + buildLoggedInMenu();
360             }
361         } catch (NumberFormatException e) {
362             message = "Invalid amount.";
363             result = "Please enter numbers only.\n\n" + buildLoggedInMenu();
364             numberPadInput = "";
365         }
366     }
}

```

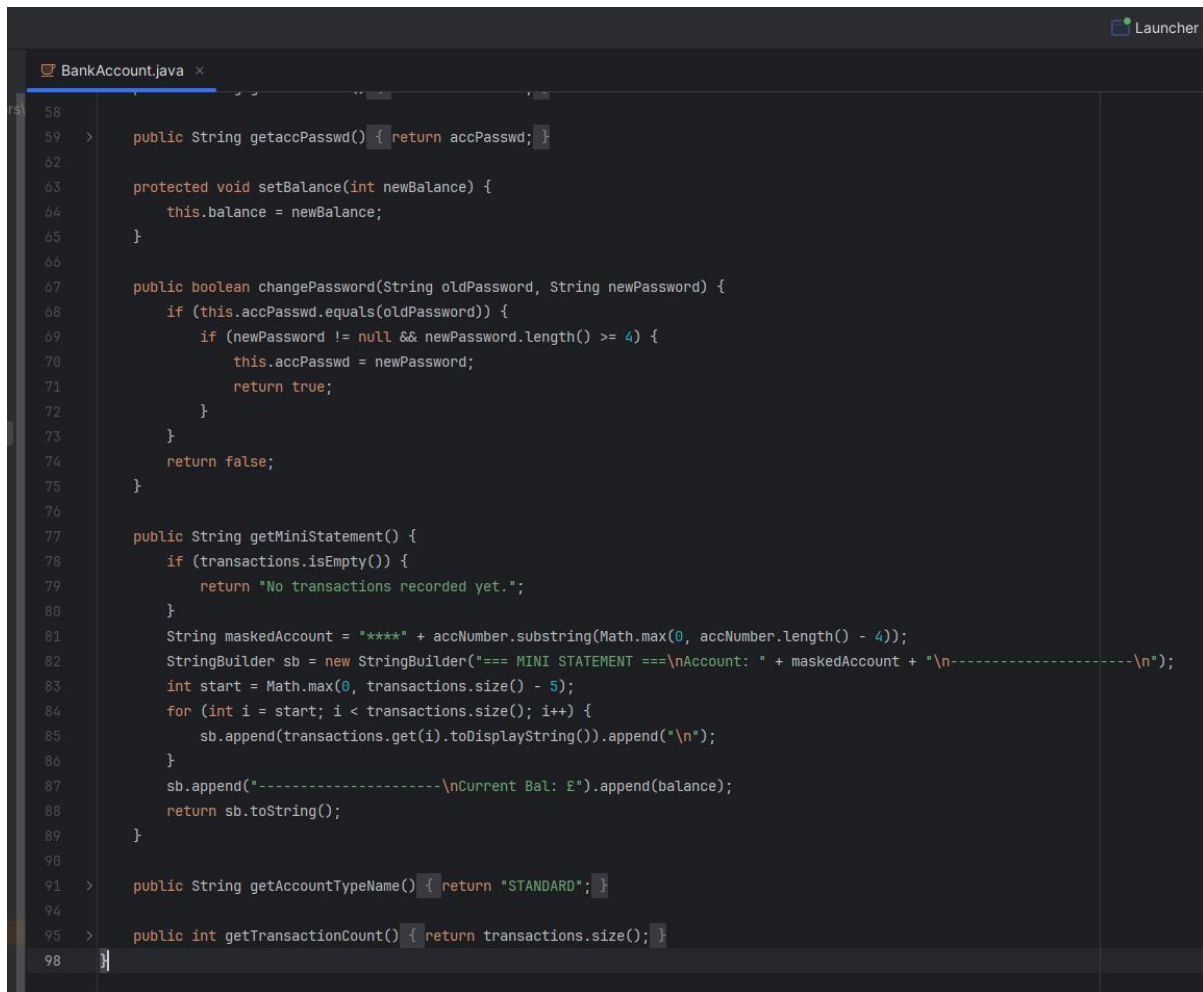
DataManager.java — serialises the entire Bank object to bank_data.ser after every transaction (autoSave) and deserialises it on startup, ensuring account balances persist between sessions. Implemented by Reema.

```

1  /**
2   * Handles saving and loading bank data to a file.
3   * Uses serialization to store Bank objects.
4   *
5   * @author Reema
6   */
7  package com.atmbanksimulator;
8
9  import java.io.*;
10
11  public class DataManager {
12      private static final String FILE_NAME = "bank_data.ser";
13
14      public static void saveBank(Bank bank) {
15          try (ObjectOutputStream oos = new ObjectOutputStream(new FileOutputStream(FILE_NAME))) {
16              oos.writeObject(bank);
17              System.out.println("Bank data saved!");
18          } catch (IOException e) {
19              System.out.println("Save error: " + e.getMessage());
20          }
21      }
22
23      public static Bank loadBank() {
24          File file = new File(FILE_NAME);
25          if (!file.exists()) {
26              return new Bank();
27          }
28          try (ObjectInputStream ois = new ObjectInputStream(new FileInputStream(FILE_NAME))) {
29              return (Bank) ois.readObject();
30          } catch (IOException | ClassNotFoundException e) {
31              System.out.println("Load error: " + e.getMessage());
32              return new Bank();
33          }
34      }
35  }

```

BankAccount.java getMiniStatement() — masks all but the last four digits of the account number (UK GDPR compliance) and returns the last five transactions from the history ArrayList.



```

58
59 > public String getaccPasswd() { return accPasswd; }
60
61
62
63 protected void setBalance(int newBalance) {
64     this.balance = newBalance;
65 }
66
67 public boolean changePassword(String oldPassword, String newPassword) {
68     if (this.accPasswd.equals(oldPassword)) {
69         if (newPassword != null && newPassword.length() >= 4) {
70             this.accPasswd = newPassword;
71             return true;
72         }
73     }
74     return false;
75 }
76
77 public String getMiniStatement() {
78     if (transactions.isEmpty()) {
79         return "No transactions recorded yet.";
80     }
81     String maskedAccount = "****" + accNumber.substring(Math.max(0, accNumber.length() - 4));
82     StringBuilder sb = new StringBuilder("=== MINI STATEMENT ===\nAccount: " + maskedAccount + "\n-----\n");
83     int start = Math.max(0, transactions.size() - 5);
84     for (int i = start; i < transactions.size(); i++) {
85         sb.append(transactions.get(i).toString()).append("\n");
86     }
87     sb.append("-----\nCurrent Bal: £").append(balance);
88     return sb.toString();
89 }
90
91 > public String getAccountTypeName() { return "STANDARD"; }
92
93
94
95 > public int getTransactionCount() { return transactions.size(); }
96
97
98 }

```

Transaction.java toString() — formats a single transaction as a fixed-width display line (date, type label, amount, running balance) for the Mini Statement screen. New class added by Reema.

```
Transaction.java x
50
51 // toDisplayString()
52 // Formats this transaction as a single readable line for the Mini Statement screen.
53 // Example output: "18/04 14:32 DEPOSIT £200 [Bal: £750]"
54 public String toDisplayString() {
55     // Format the timestamp to just show date and time (no year, saves screen space)
56     DateTimeFormatter fmt = DateTimeFormatter.ofPattern("dd/MM HH:mm");
57     String timeStr = timestamp.format(fmt);
58
59     // Short labels for each transaction type
60     String typeLabel = switch (type) {
61         case DEPOSIT -> "DEPOSIT ";
62         case WITHDRAWAL -> "WITHDRAW ";
63         case TRANSFER_OUT -> "TRF OUT ";
64         case TRANSFER_IN -> "TRF IN ";
65         case INTEREST -> "INTEREST ";
66         case OVERDRAFT_FEE -> "OVD FEE ";
67     };
68
69     // Build the full display line, padding for column alignment
70     return String.format("%s %s +/-£%-6d Bal:£%d",
71         timeStr, typeLabel, amount, balanceAfter);
72 }
73
74
75 // GETTERS – Read-only access to transaction data
76 // These are used by getMiniStatement() in BankAccount
77
78 /** Returns the type of transaction (DEPOSIT, WITHDRAWAL, etc.) */
79 > public TransactionType getType() { return type; }
82
83 /** Returns the amount of money involved in this transaction */
84 > public int getAmount() { return amount; }
87
88 /** Returns the account balance immediately after this transaction */
89 > public int getBalanceAfter() { return balanceAfter; }
92
93 /** Returns the exact date and time this transaction was recorded */
94 > public LocalDateTime getTimestamp() { return timestamp; }
97 }
```

main > java > com > atmbanksimulator > Transaction.java